United States Patent Application Publication	20250284662
Kind Code	A1
Publication Date	September 11, 2025
Inventor(s)	Krishnamoorthy; Boopathy et al.

CROSS-NODE FILE SYSTEM CONTEXT CHECKS WITHIN A DISTRIBUTED STORAGE SYSTEM USING DISAGGREGATED STORAGE

Abstract

Systems and methods for implementing context checks that account for the potential for file movement across nodes of a cluster of a distributed storage system are provided. Context data utilized for performing context checking in connection with performing read operations may include a buffer tree identifier (bufftree ID), a data ID, and an epoch in which the bufftree ID represents a volume ID that is unique across the cluster, the data ID corresponds to a file block number within the file at issue from which data is being read, and the epoch is a value that facilitates cluster-wide timeline checks. In one embodiment, the bufftree ID may be ensured to be unique across the cluster by, during the process of creating a new volume, combining a unique ID of the DEFS hosting the new volume with a monotonically increasing volume count maintained for the DEFS.

Inventors: **Krishnamoorthy; Boopathy (Bangalore, IN), Makam; Sumith (Bangalore, IN), Gupta; Saurabh (Haryana, IN), Subramanian; Ananthan (San Ramon, CA), Thoppil; Anil Paul (Pleasanton, CA), Maharana; Pragyan Anand (Bengaluru, IN)**

Applicant: **NetApp, Inc.** (San Jose, CA)

Family ID: **1000008573145**

Assignee: **NetApp, Inc. (San Jose, CA)**

Appl. No.: **19/183991**

Filed: **April 21, 2025**

Foreign Application Priority Data

IN	202441062849	Aug. 20, 2024

Related U.S. Application Data

parent US continuation-in-part 18595785 20240305 PENDING child US 19183991

Publication Classification

Int. Cl.: **G06F16/16** (20190101); **G06F16/182** (20190101)

U.S. Cl.:

CPC **G06F16/16** (20190101); **G06F16/182** (20190101);

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation-in-part of U.S. patent application Ser. No. 18/595,785, filed on Mar. 5, 2024 and claims the benefit of priority of Indian Provisional Application No. 202441062849, filed on Aug. 20, 2024, both of which are hereby incorporated by reference in their entirety for all purposes.

BACKGROUND
Field
[0002] Various embodiments of the present disclosure generally relate to storage systems. In particular, some embodiments relate to the implementation and use of context checks in a distributed storage system having a disaggregated storage architecture in which volumes may be moved from one dynamically extensible file system (DEFS) to another and/or in which allocation areas (AAs) created within a disaggregated storage space shared by the nodes of the distributed storage system as well as bitmaps associated with the AAs may be moved from one DEFS to another.
Description of the Related Art
[0003] Many storage systems store a checksum value with each data block (e.g., a count of the number of set bits in the data block). In this manner, when reading the data block, the checksum may be confirmed to ensure that the data block was read correctly, for example, by comparing a newly computed checksum based on the read data against the stored checksum. Furthermore, certain storage systems are configured to store context information (or “context data”) with each data block in addition to the checksum. For instance, certain storage file systems (e.g., the Write Anywhere File Layout (WAFL®) file system (available from NetApp, Inc. of San Jose, CA)) may implement various techniques to point to physical storage locations for data block access. As such, while the checksum may be used to confirm that the data within a stored data block was read correctly, performing a “context check” based on the context data (e.g., represented by a tuple including a buffer tree identifier (ID) (which may also be referred to herein as a “bufftree ID”), a data ID or file block number (FBN), and information indicative of a relative write time of the data at issue) may be used to confirm that the data block accessed is the correct data block. As will be appreciated by those skilled in the art, various scenarios in which the data block accessed may not be the correct data block include situations in which the data block has either been incorrectly written (e.g., a “lost write”) or the data block has been moved (“reallocated”) to a new physical location (e.g., to defragment free space).
SUMMARY
[0004] Systems and methods are described for implementation and use context checks that account

for the potential for file movement across nodes of a cluster of a distributed storage system that makes use of disaggregated storage. According to one embodiment, a read request from a requestor to read data from a file is received by a node of multiple nodes of a cluster representing a distributed storage system. An expected set of context data associated with the read request is determined in which the expected set of context data includes at least a first buffer tree identifier (bufftree ID) associated with the file that is unique across the cluster and a current epoch value of a dynamically extensible file system (DEFS) in which the file is stored. Prior to returning a block of data of the requested data to the requestor, at least the following conditions are verified: (i) a second bufftree ID contained within a second set of context data associated with the block of data matches the first bufftree ID in the expected set of context data; and (ii) an epoch value contained within the second set of context data and the current epoch value of the expected set of context data satisfy a cluster-wide timeline check.

[0005] Other features of embodiments of the present disclosure will be apparent from accompanying drawings and detailed description that follows.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0006] In the Figures, similar components and/or features may have the same reference label. Further, various components of the same type may be distinguished by following the reference label with a second label that distinguishes among the similar components. If only the first reference label is used in the specification, the description is applicable to any one of the similar components having the same first reference label irrespective of the second reference label.

[0007] FIG. **1** is a block diagram illustrating a plurality of nodes interconnected as a cluster in accordance with an embodiment of the present disclosure.

[0008] FIG. **2** is a block diagram illustrating a node in accordance with an embodiment of the present disclosure.

[0009] FIG. **3** is a block diagram illustrating a storage operating system in accordance with an embodiment of the present disclosure.

[0010] FIG. **4** is a block diagram illustrating a tree of blocks representing of an example a file system layout in accordance with an embodiment of the present disclosure.

[0011] FIG. **5** is a block diagram illustrating a distributed storage system architecture in which the entirety of a given disk and a given RAID group are owned by an aggregate and the aggregate file system is only visible from one node, thereby resulting in silos of storage space.

[0012] FIG. **6**A is a block diagram illustrating a distributed storage system architecture that provides disaggregated storage in accordance with an embodiment of the present disclosure.

[0013] FIG. **6**B is a high-level flow diagram illustrating operations for establishing disaggregated storage within a storage pod in accordance with an embodiment of the present disclosure.

[0014] FIG. **7** is a block diagram conceptually illustrating movement of a volume as well as AA movement and associated metafile movement from a source DEFS to a destination DEFS in accordance with an embodiment of the present disclosure.

[0015] FIG. **8**A is a block diagram illustrating a data block having context data that may be stored in a physical storage location in accordance with an embodiment of the present disclosure.

[0016] FIG. **8**B is a block diagram illustrating an example of a bufftree ID for a file in accordance with an embodiment of the present disclosure.

[0017] FIG. **8**C is a block diagram illustrating an example of a bufftree ID for a metafile in accordance with an embodiment of the present disclosure.

[0018] FIG. **9**A is a flow diagram illustrating operations for performing volume creation in accordance with an embodiment of the present disclosure

[0019] FIG. **9**B is a flow diagram illustrating operations for performing file creation in accordance with an embodiment of the present disclosure.
[0020] FIG. **10** is a flow diagram illustrating operations for performing context checking in accordance with an embodiment of the present disclosure.

DETAILED DESCRIPTION

[0021] Systems and methods are described for implementation and use context checks that account for the potential for file movement across nodes of a cluster of a distributed storage system that makes use of disaggregated storage. Distributed storage systems generally take the form of a cluster of storage controllers (or nodes in virtual or physical form). Some prior scale-out storage solutions tightly couple compute and storage infrastructure. For example, as shown in FIG. **5**, each node of a distributed storage system may be associated with a dedicated pool of storage space (e.g., a node-level aggregate representing a file system that holds one or more volumes created over one or more RAID groups and which is only accessible from a single node at a time). As described further below, in such an environment, various portions of context data generally need only be unique within the node-level aggregate. However, when moving to a scale out storage solution architecture, for example, as described with reference to FIG. **6**A, storage space may be used more fluidly across all the individual storage systems (e.g., nodes) of a distributed storage system (e.g., a cluster of nodes working together). While this architecture provides many advantages, it introduces additional complexity in regards to performing context checking. For example, in contrast to the entirety of a given storage device (e.g., a disk) being owned by a node-level aggregate and the aggregate file system being visible from only one node of a cluster as shown and described with reference to FIG. **5**, the use of "dynamically extensible file systems" in the context of FIG. **6**A facilitates visibility by all nodes in the cluster to the entirety of a global physical volume block number (PVBN) space of the storage devices associated with a single "storage pod" that may be shared by all of the nodes of the cluster with space from the global PVBN space being used on demand.
[0022] The use of a common storage pod shared by all nodes of the storage cluster facilitates certain features, for example, efficient volume movement from one DEFS to another DEFS on the same node or on a different node of the cluster. Non-limiting examples of an efficient volume movement operation (e.g., a zero-copy volume move) that may be performed in a distributed storage system that makes use of disaggregated storage is described in U.S. Pat. No. 12,204,784, which is hereby incorporated by reference in its entirety for all purposes.
[0023] As described further below, in some examples, the association of blocks to a dynamically extensible file system may be in large chunks of one or more gigabytes (GB), which are referred to herein as "allocation areas" (AAs) that each include multiple RAID stripes. In addition to the potential for volume movement (and hence, movement of all files associated with the volume at issue), these AAs (and associated metadata information, for example, in the form of metafiles or portions thereof) may be moved from one dynamically extensible file system (DEFS) to another on the same or a different node of the cluster to facilitate space balancing. A non-limiting example of efficient movement of metadata information associated with AAs being moved, for example, during performance of space balancing is described in co-pending U.S. patent application Ser. No. 19/068,324, filed on Mar. 3, 1925, which is hereby incorporated by reference in its entirety for all purposes. As a result of the ability for AAs and their associated metadata information to be moved from one DEFS to another and the ability to readily move a volume from one DEFS to another, prior assumptions regarding certain portions (e.g., the bufftree ID) of the context data needing to only be unique within a given node (or node-level aggregate) no longer hold true. Additionally, in view of the fact that (i) files and/or metafiles (or portions thereof) may be moved among DEFSs of the cluster (e.g., as a result of performance of a volume move operation and/or as a result of moving AAs to balance space among the DEFSs) and (ii) CPs may be performed independently by DEFSs within the cluster, a timing sanity check (e.g., the data was not written in the future) that

relies on a CP count maintained by individual DEFSs to provide information regarding a write time of the data at issue is no longer reliable.

[0024] Embodiments described herein introduce new context data for use in connection with performing context checking during a read operation. The new context data precludes bufftree ID collisions, for example, when a file is moved from one DEFS to another as part of a volume move operation and may be in the form of a tuple (e.g., <bufftree ID, data ID, epoch>) in which the bufftree ID represents a volume ID that is unique across the cluster, the data ID corresponds to an FBN within the file at issue from which data is being read, and the epoch is a value that facilitates cluster-wide timeline checks. As described further below, in one embodiment, the bufftree ID may be ensured to be unique across the cluster by, during the process of creating a new volume, combining a unique DEFS ID of the DEFS hosting the new volume with a monotonically increasing volume count maintained for the DEFS. In this manner, when a volume is moved from one DEFS to another, there will be no potential for bufftree ID collisions. As described further below, when AA movement or volume movement is performed from a source DEFS to a destination DEFS, a current epoch value maintained by the destination DEFS is updated to ensure it is ahead of a current epoch value independently maintained by the source DEFS to ensure proper functioning of the timeline check associated with context checking.

[0025] In the following description, numerous specific details are set forth in order to provide a thorough understanding of embodiments of the present disclosure. It will be apparent, however, to one skilled in the art that embodiments of the present disclosure may be practiced without some of these specific details. In other instances, well-known structures and devices are shown in block diagram form.

Terminology

[0026] Brief definitions of terms used throughout this application are given below.

[0027] The terms “connected” or “coupled” and related terms are used in an operational sense and are not necessarily limited to a direct connection or coupling. Thus, for example, two devices may be coupled directly, or via one or more intermediary media or devices. As another example, devices may be coupled in such a way that information can be passed there between, while not sharing any physical connection with one another. Based on the disclosure provided herein, one of ordinary skill in the art will appreciate a variety of ways in which connection or coupling exists in accordance with the aforementioned definition.

[0028] If the specification states a component or feature “may”, “can”, “could”, or “might” be included or have a characteristic, that particular component or feature is not required to be included or have the characteristic.

[0029] The terms “component”, “module”, “system,” and the like as used herein are intended to refer to a computer-related entity, either software-executing general-purpose processor, hardware, firmware and a combination thereof. For example, a component may be, but is not limited to being, a process running on a hardware processor, a hardware processor, an object, an executable, a thread of execution, a program, and/or a computer. By way of illustration, both an application running on a server and the server can be a component. One or more components may reside within a process and/or thread of execution, and a component may be localized on one computer and/or distributed between two or more computers. Also, these components can be executed from various computer readable media having various data structures stored thereon. The components may communicate via local and/or remote processes such as in accordance with a signal having one or more data packets (e.g., data from one component interacting with another component in a local system, distributed system, and/or across a network such as the Internet with other systems via the signal).

[0030] The term file/files as used herein include data container/data containers, directory/directories, and/or data object/data objects with structured or unstructured data. Some files may be used to store client data and other files (e.g., metafiles) may be used to store metadata used by the storage operating system or a DEFS (e.g., a space map indicative of which PVBNs

within a storage pod are in use or an active map indicative of which PVBNs of AAs owned by a given DEFS are in use).

[0031] As used in the description herein and throughout the claims that follow, the meaning of “a,” “an,” and “the” includes plural reference unless the context clearly dictates otherwise. Also, as used in the description herein, the meaning of “in” includes “in” and “on” unless the context clearly dictates otherwise.

[0032] The phrases “in an embodiment,” “according to one embodiment,” and the like generally mean the particular feature, structure, or characteristic following the phrase is included in at least one embodiment of the present disclosure and may be included in more than one embodiment of the present disclosure. Importantly, such phrases do not necessarily refer to the same embodiment.

[0033] As used herein a “cloud” or “cloud environment” broadly and generally refers to a platform through which cloud computing may be delivered via a public network (e.g., the Internet) and/or a private network. The National Institute of Standards and Technology (NIST) defines cloud computing as “a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction.” P. Mell, T. Grance, The NIST Definition of Cloud Computing, National Institute of Standards and Technology, USA, 2011. The infrastructure of a cloud may be deployed in accordance with various deployment models, including private cloud, community cloud, public cloud, and hybrid cloud. In the private cloud deployment model, the cloud infrastructure is provisioned for exclusive use by a single organization comprising multiple consumers (e.g., business units), may be owned, managed, and operated by the organization, a third party, or some combination of them, and may exist on or off premises. In the community cloud deployment model, the cloud infrastructure is provisioned for exclusive use by a specific community of consumers from organizations that have shared concerns (e.g., mission, security requirements, policy, and compliance considerations), may be owned, managed, and operated by one or more of the organizations in the community, a third party, or some combination of them, and may exist on or off premises. In the public cloud deployment model, the cloud infrastructure is provisioned for open use by the general public, may be owned, managed, and operated by a cloud provider or hyperscaler (e.g., a business, academic, or government organization, or some combination of them), and exists on the premises of the cloud provider. The cloud service provider may offer a cloud-based platform, infrastructure, application, or storage services as-a-service, in accordance with a number of service models, including Software-as-a-Service (SaaS), Platform-as-a-Service (PaaS), and/or Infrastructure-as-a-Service (IaaS). In the hybrid cloud deployment model, the cloud infrastructure is a composition of two or more distinct cloud infrastructures (private, community, or public) that remain unique entities, but are bound together by standardized or proprietary technology that enables data and application portability (e.g., cloud bursting for load balancing between clouds).

[0034] As used herein, a “storage system” or “storage appliance” generally refers to a type of computing appliance or node, in virtual or physical form, that provides data to, or manages data for, other computing devices or clients (e.g., applications). The storage system may be part of a cluster of multiple nodes representing a distributed storage system. In various examples described herein, a storage system may be run (e.g., on a VM or as a containerized instance, as the case may be) within a public cloud provider.

[0035] As used herein, the term “storage operating system” generally refers to computer-executable code operable on a computer to perform a storage function that manages data access and may, in the case of a storage system (e.g., a node), implement data access semantics of a general purpose operating system. The storage operating system can also be implemented as a microkernel, an application program operating over a general-purpose operating system, such as UNIX or Windows NT, or as a general-purpose operating system with configurable functionality, which is configured

for storage applications as described herein. In some embodiments, a light-weight data adaptor may be deployed on one or more server or compute nodes added to a cluster to allow compute-intensive data services to be performed without adversely impacting performance of storage operations being performed by other nodes of the cluster. The light-weight data adaptor may be created based on a storage operating system but, since the server node will not participate in handling storage operations on behalf of clients, the light-weight data adaptor may exclude various subsystems/modules that are used solely for serving storage requests and that are unnecessary for performance of data services. In this manner, compute intensive data services may be handled within the cluster by one of more dedicated compute nodes.

[0036] As used herein, a “cloud volume” generally refers to persistent storage that is accessible to a virtual storage system by virtue of the persistent storage being associated with a compute instance in which the virtual storage system is running. A cloud volume may represent a hard-disk drive (HDD) or a solid-state drive (SSD) from a pool of storage devices (or “disks” which is used interchangeably throughout this specification) within a cloud environment that is connected to the compute instance through Ethernet or fibre channel (FC) switches as is the case for network-attached storage (NAS) or a storage area network (SAN). Non-limiting examples of cloud volumes include various types of SSD volumes (e.g., AWS Elastic Block Store (EBS) gp2, gp3, io1, and io2 volumes for EC2 instances) and various types of HDD volumes (e.g., AWS EBS st1 and sc1 volumes for EC2 instances).

[0037] As used herein a “consistency point” or “CP” generally refers to the act of writing data to disk and updating active file system pointers. In various examples, when a file system of a storage system receives a write request, it commits the data to permanent storage before the request is confirmed to the writer. Otherwise, if the storage system were to experience a failure with data only in volatile memory, that data would be lost, and underlying file structures could become corrupted. Physical storage appliances commonly use battery-backed high-speed non-volatile random access memory (NVRAM) as a journaling storage media to journal writes and accelerate write performance while providing permanence, because writing to memory is much faster than writing to storage (e.g., disk). Storage systems may also implement a buffer cache in the form of an in-memory cache to cache data that is read from data storage media (e.g., local mass storage devices or a storage array associated with the storage system) as well as data modified by write requests. In this manner, in the event a subsequent access relates to data residing within the buffer cache, the data can be served from local, high performance, low latency storage, thereby improving overall performance of the storage system. Virtual storage appliances may use NV storage backed by cloud volumes in place of NVRAM for journaling storage and for the buffer cache. Regardless of whether NVRAM or NV storage is utilized, the modified data may be periodically (e.g., every few seconds) flushed to the data storage media. As the buffer cache may be limited in size, an additional cache level may be provided by a victim cache, typically implemented within a slower memory or storage device than utilized by the buffer cache, that stores data evicted from the buffer cache. The event of saving the modified data to the mass storage devices may be referred to as a CP. At a CP, the file system may save any data that was modified by write requests to persistent data storage media. As will be appreciated, when using a buffer cache, there is a small risk of a system failure occurring between CPs, causing the loss of data modified after the last CP. Consequently, the storage system may maintain an operation log or journal of certain storage operations within the journaling storage media that have been performed since the last CP. This log may include a separate journal entry (e.g., including an operation header) for each storage request received from a client that results in a modification to the file system or data. Such entries for a given file may include, for example, “Create File,” “Write File Data,” and the like. Depending upon the operating mode or configuration of the storage system, each journal entry may also include the data to be written according to the corresponding request. The journal may be used in the event of a failure to recover data that would otherwise be lost. For example, in the event of a

failure, it may be possible to replay the journal to reconstruct the current state of stored data just prior to the failure. As described further below, in various examples there may be one or more predefined or configurable triggers (CP triggers). Responsive to a given CP trigger (or at a CP), the file system may save any data that was modified by write requests to persistent data storage media.
[0038] As used herein, a “buffer tree identifier,” a “buffer tree ID,” a “bufftree ID” or the like generally refers to an identifier associated with a volume that is unique cluster-wide. Use of such bufftree IDs ensures when volumes are moved from one DEFS to another there will be no collisions among bufftree IDs, for example, that would create false positives in connection with context checking performed when reading from files of the volumes.
[0039] As used herein, a “consistency point count” or “CP count” generally refers to a count of the number of CPs that have been performed by a given DEFS of a cluster. The CP count may be used to perform local timeline checks, but as noted above, may not be used to perform cluster-wide timeline checks due to the fact that the DEFSs of a cluster independently perform CPs and may do so at different rates.
[0040] As used herein, an “epoch value” or simply an “epoch” generally refers to a numeric value representing a specific point in an ever-growing timeline across all DEFSs of a cluster and which may be used to enable cluster-wide timeline comparisons.
[0041] As used herein, a “lost write” generally refers to a situation in which a storage system acknowledges a write as completed, but the effects of that write are no longer present after some time, meaning the data was not actually persisted to storage.
[0042] As used herein, a “RAID stripe” generally refers to a set of blocks spread across multiple storage devices (e.g., disks of a disk array, disks of a disk shelf, or cloud volumes) to form a parity group (or RAID group).
[0043] As used herein, an “allocation area” or “AA” generally refers to a group of RAID stripes. In various examples described herein a single storage pod may be shared by a distributed storage system by assigning ownership of AAs to respective dynamically extensible file systems of a storage system.
[0044] As used herein, “ownership” of an AA generally refers to the ability of the owning DEFS to use the AA space (e.g., the blocks associated with the AA) for performance of writes or write operations. In the context of various embodiments described herein, only one DEFS can write to a given block (PVBN) at a time for multiple correctness reasons, so it is the DEFS that owns the given AA of which the given block is associated that has the exclusive ability among all other DEFSs in the storage system to write to the given block. Further, in embodiments described herein, for the file system metadata to be correct, the file system metadata for a given AA is coordinated in one place.
[0045] As used herein, a “free allocation area” or “free AA” generally refers to an AA in which no PVBNs of the AA are marked as used, for example, by any active maps of a given dynamically extensible file system.
[0046] As used herein, a “partial allocation area” or “partial AA” generally refers to an AA in which one or more PVBNs of the AA are marked as in use (containing valid data), for example, by an active map of a given dynamically extensible file system. As discussed further below, in connection with space balancing, while it is preferable to perform AA ownership changes of free AAs, in various examples, space balancing may involve one dynamically extensible file system donating one or more partial AAs to another dynamically extensible file system. In such cases, the additional cost of copying portions of one or more associated data structures (e.g., bit maps, such as an active map, a refcount map, a summary map, an AA information map, and a space map) relating to storage space information may be incurred. No such additional cost is incurred when moving or changing ownership of free AAs. These associated data structures may, among other things, track which PVBNs are in use, track PVBN counts per AA (e.g., total used blocks and shared references to blocks) and other flags.

[0047] As used herein, a “storage pod” generally refers to a group of disks containing multiple RAID groups that are accessible from all storage systems (nodes) of a distributed storage system (cluster).
[0048] As used herein, a “data pod” generally refers to a set of storage systems (nodes) that share the same storage pod. In some examples, a data pod refers to a single cluster of nodes representing a distributed storage system. In other examples, there can be multiple data pods in a cluster. Data pods may be used to limit the fault domain and there can be multiple HA pairs of nodes within a data pod.
[0049] As used herein, an “active map” is a data structure that contains information indicative of which PVBNs of a distributed file system are in use. In one embodiment, the active map is represented in the form of a sparce bit map, for example, maintained within a metafile, in which each PVBN of a global PVBN space of a storage pod has a corresponding Boolean value (or truth value) represented as a single bit, for example, in which the true (**1**) indicates the corresponding PVBN is in use and false (**0**) indicates the corresponding PVBN is not in use.
[0050] As used herein, a “dynamically extensible file system” or a “DEFS” generally refers to a file system of a data pod or a cluster that has visibility into the entire global PVBN space of a storage pod and hosts multiple volumes. A DEFS may be thought of as a data container or a storage container (which may be referred to as a storage segment container) to which AAs are assigned, thereby resulting in a more flexible and enhanced version of a node-level aggregate. As described further herein (for example, in connection with automatic space balancing), the storage space associated with one or more AAs of a given DEFS may be dynamically transferred or moved on demand to any other DEFS in the cluster by changing the ownership of the one or more AAs and moving associated AA tracking data structures as appropriate. This provides the unique ability to independently scale each DEFS of a cluster. For example, DEFSs can shrink or grow dynamically over time to meet their respective storage needs and silos of storage space are avoided. In one embodiment, a distributed file system comprises multiple instances of the WAFL® Copy-on-Write file system running on respective storage systems (nodes) of a distributed storage system (cluster) that represents the data pod. In various examples described herein, a given storage system (node) of a distributed storage system (cluster) may own one or more DEFSs including, for example, a log DEFS for hosting an operation log or journal of certain storage operations that have been performed by the node since the last CP and a data DEFS for hosting customer volumes or logical unit numbers (LUNs). As described further below, the partitioning/division of a storage pod into AAs (creation of a disaggregated storage space) and the distribution of ownership of AAs among DEFSs of multiple nodes of a cluster may facilitate implementation of a distributed storage system having a disaggregated storage architecture. In various examples described herein, each storage system may have its own portion of disaggregated storage to which it has the exclusive ability to perform write access, thereby simplifying storage management by, among otherings, not requiring implementation of access control mechanisms, for example, in the form of locks. At the same time, each storage system also has visibility into the entirety of a global PVBN space, thereby allowing read access by a given storage system to any portion of the disaggregated storage regardless of which node of the cluster is the current owner of the underlying allocation areas. Based disclosure provided herein, those skilled in the art will understand there are at least two types of disaggregation represented/achieved within various examples, including (i) the disaggregation of storage space provided by a storage pod by dividing or partitioning the storage space into AAs the ownership of which can be fluidly changed from one DEFS to another on demand and (ii) the disaggregation of the storage architecture into independent components, including the decoupling of processing resources and storage resources, thereby allowing them to be independently scaled. In one embodiment, the former (which may also be referred to as modular storage, partitioned storage, adaptable storage, or fluid storage) facilitates the latter.
[0051] As used herein, an “allocation area map” or “AA map” generally refers to a per dynamically

extensible file system data structure or file (e.g., a metafile) that contains information at an AA-level of granularity indicative of which AAs are assigned to or "owned" by a given dynamically extensible file system.

[0052] A "node-level aggregate" generally refers to a file system of a single storage system (node) that holds multiple volumes created over one or more RAID groups, in which the node owns the entire PVBN space of the collection of disks of the one or more RAID groups. Node-level aggregates are only accessible from a single storage system (node) of a distributed storage system (cluster) at a time.

[0053] As used herein, an "inode" generally refers to a file data structure maintained by a file system that stores metadata for data containers (e.g., directories, subdirectories, disk files, etc.). An inode may include, among other things, location, file size, permissions needed to access a given file with which it is associated as well as creation, read, and write timestamps, and one or more flags.

[0054] As used herein, a "storage volume" or "volume" generally refers to a container in which applications, databases, and file systems store data. A volume is a logical component created for the host to access storage on a storage array. A volume may be created from the capacity available in storage pod, a pool, or a volume group. A volume has a defined capacity. Although a volume might consist of more than one drive, a volume appears as one logical component to the host. Non-limiting examples of a volume include a flexible volume and a flexgroup volume.

[0055] As used herein, a "flexible volume" generally refers to a type of storage volume that may be efficiently distributed across multiple storage devices. A flexible volume may be capable of being resized to meet changing business or application requirements. In some embodiments, a storage system may provide one or more aggregates and one or more storage volumes distributed across a plurality of nodes interconnected as a cluster. Each of the storage volumes may be configured to store data such as files and logical units. As such, in some embodiments, a flexible volume may be comprised within a storage aggregate and further comprises at least one storage device. The storage aggregate may be abstracted over a RAID plex where each plex comprises a RAID group. Moreover, each RAID group may comprise a plurality of storage disks. As such, a flexible volume may comprise data storage spread over multiple storage disks or devices. A flexible volume may be loosely coupled to its containing aggregate. A flexible volume can share its containing aggregate with other flexible volumes. Thus, a single aggregate can be the shared source of all the storage used by all the flexible volumes contained by that aggregate. A non-limiting example of a flexible volume is a NetApp ONTAP FlexVol volume.

[0056] As used herein, a "flexgroup volume" generally refers to a single namespace that is made up of multiple constituent/member volumes. A non-limiting example of a flexgroup volume is a NetApp ONTAP FlexGroup volume that can be managed by storage administrators, and which acts like a NetApp FlexVol volume. In the context of a flexgroup volume, "constituent volume" and "member volume" are interchangeable terms that refer to the underlying volumes (e.g., flexible volumes) that make up the flexgroup volume.

Example Distributed Storage System Cluster

[0057] FIG. **1** is a block diagram illustrating a plurality of nodes **110***a-b* interconnected as a cluster **100** in accordance with an embodiment of the present disclosure. In the context of the present example, the nodes **110***a-b* comprise various functional components that cooperate to provide a distributed storage system architecture of the cluster **100**. To that end, in the context of the present example, each node is generally organized as a network element (e.g., network element **120***a* or **120***b*) and a disk element (e.g., disk element **150***a* or **150***b*). The network element includes functionality that enables the node to connect to clients (e.g., client **180**) over a computer network **140**, while each disk element **350** connects to one or more storage devices, such as disks, of one or more disk arrays (not shown) or of one or more storage shelves (not shown), represented as a single shared storage pod **145**.

[0058] In the context of the present example, the nodes **110***a-b* are interconnected by a cluster

switching fabric **151** which, in an example, may be embodied as a Gigabit Ethernet switch. It should be noted that while there is shown an equal number of network and disk elements in the illustrative cluster **100**, there may be differing numbers of network and/or disk elements. For example, there may be a plurality of network elements and/or disk elements interconnected in a cluster configuration **100** that does not reflect a one-to-one correspondence between the network and disk elements. As such, the description of a node comprising one network element and one disk element should be taken as illustrative only.

[0059] Clients may be general-purpose computers configured to interact with the node in accordance with a client/server model of information delivery. That is, each client (e.g., client **180**) may request the services of the node, and the node may return the results of the services requested by the client, by exchanging packets over the network **140**. The client may issue packets including file-based access protocols (e.g., the Common Internet File System (CIFS) protocol or Network File System (NFS) protocol), over the Transmission Control Protocol/Internet Protocol (TCP/IP) when accessing information in the form of files and directories. Alternatively, the client may issue packets including block-based access protocols, such as the Small Computer Systems Interface (SCSI) protocol encapsulated over TCP (iSCSI) and SCSI encapsulated over Fibre Channel (FCP), when accessing information in the form of blocks. In various examples described herein, an administrative user (not shown) of the client may make use of a user interface (UI) presented by the cluster or a command line interface (CLI) of the cluster to, among other things, establish a data protection relationship between a source volume and a destination volume (e.g., a mirroring relationship specifying one or more policies associated with creation, retention, and transfer of snapshots), defining snapshot and/or backup policies, and association of snapshot policies with snapshots.

[0060] Storage elements (e.g., disk elements **150***a* and **150***b*) are illustratively connected to storage devices (e.g., disks) (not shown) within that may be organized into storage (disk) arrays within the storage pod **145**. Alternatively, storage devices other than disks may be utilized, e.g., flash memory, optical storage, solid state devices, etc. As such, the description of disks should be taken as exemplary only and references to disks herein should be understood to refer to storage devices more generally.

[0061] In general, various embodiments envision a cluster (e.g., cluster **100**) in which every node (e.g., nodes **110***a*-*b*) can essentially talk to every storage device (e.g., disk) in the storage pod **145**. This is in contrast to the distributed storage system architecture described with reference to FIG. **5**. In examples described herein, all nodes (e.g., nodes **110***a*-*b*) of the cluster have visibility and read access to an entirety of a global PVBN space of the storage pod **145**, for example, via an interconnect layer **142**. As described further below, according to one embodiment, the storage within the storage pod **145** is grouped into distinct allocation areas (AAs) than can be assigned to a given dynamically extensible file system (DEFS) of a node to facilitate implementation disaggregated storage. In examples described herein, the AAs assigned to a given DEFS may be said to “own” the assigned AAs and the node owning the given DEFS has the exclusive write access to the associated PVBNs and the exclusive ability to perform write allocation from such blocks. In one embodiment, each node has its own view of a portion of the disaggregated storage represented by the assignment of, for example, via respective allocation area (AA) maps and active maps. This granular assignment of AAs and ability to fluidly change ownership of AAs as needed facilitates the elimination of per-node storage silos and provides higher and more predictable performance, which further translate into improved storage utilization and improvements in cost effectiveness of the storage solution.

[0062] Depending on the particular implementation, the interconnect layer **142** may be represented by an intermediate switching topology or some other interconnectivity layer or disk switching layer between the disks in the storage pod **145** and the nodes. Non-limiting examples of the interconnect layer **150** include one or more fiber channel switches or one or more non-volatile memory express

(NVMe) fabric switches. Additional details regarding the storage pod **145**, DEFSs, AA maps, active maps, and the use, ownership, and sharing (transferring of ownership) of AAs are described further below.

Example Storage System Node

[0063] FIG. **2** is a block diagram of a node **200** that is illustratively embodied as a storage system comprising a plurality of processors (e.g., processors **222***a-b*), a memory **224**, a network adapter **225**, a cluster access adapter **226**, a storage adapter **228** and local storage **230** interconnected by a system bus **223**. Node **200** may be analogous to nodes **110***a* and **110***b* of FIG. **1**. The local storage **230** comprises one or more storage devices, such as disks, utilized by the node to locally store configuration information (e.g., in configuration table **235**). The cluster access adapter **226** comprises a plurality of ports adapted to couple the node **200** to other nodes of the cluster (e.g., cluster **100**). Illustratively, Ethernet is used as the clustering protocol and interconnect media, although it will be apparent to those skilled in the art that other types of protocols and interconnects may be utilized within the cluster architecture described herein. Alternatively, where the network elements and disk elements are implemented on separate storage systems or computers, the cluster access adapter **226** is utilized by the network and disk element for communicating with other network and disk elements in the cluster.

[0064] In the context of the present example, each node **200** is illustratively embodied as a dual processor storage system executing a storage operating system **210** that implements a high-level module, such as a file system, to logically organize the information as a hierarchical structure of named directories, files and special types of files called virtual disks (hereinafter generally "blocks") on the disks. However, it will be apparent to those of ordinary skill in the art that the node **200** may alternatively comprise a single or more than two processor system. Illustratively, one processor (e.g., processor **222***a*) may execute the functions of the network element (e.g., network element **120***a* or **120***b*) on the node, while the other processor (e.g., processor **222***b*) may execute the functions of the disk element (e.g., disk element **150***a* or **150***b*).

[0065] The memory **224** illustratively comprises storage locations that are addressable by the processors and adapters for storing software program code and data structures associated with the subject matter of the disclosure. The processor and adapters may, in turn, comprise processing elements and/or logic circuitry configured to execute the software code and manipulate the data structures. The storage operating system **210**, portions of which is typically resident in memory and executed by the processing elements, functionally organizes the node **200** by, inter alia, invoking storage operations in support of the storage service implemented by the node. It will be apparent to those skilled in the art that other processing and memory means, including various computer readable media, may be used for storing and executing program instructions pertaining to the disclosure described herein.

[0066] The network adapter **225** comprises a plurality of ports adapted to couple the node **200** to one or more clients (e.g., client **180**) over point-to-point links, wide area networks, virtual private networks implemented over a public network (Internet) or a shared local area network. The network adapter **225** thus may comprise the mechanical, electrical and signaling circuitry needed to connect the node to a network (e.g., computer network **140**). Illustratively, the network may be embodied as an Ethernet network or a Fibre Channel (FC) network. Each client (e.g., client **180**) may communicate with the node over network by exchanging discrete frames or packets of data according to pre-defined protocols, such as TCP/IP.

[0067] The storage adapter **228** cooperates with the storage operating system **210** executing on the node **200** to access information requested by the clients. The information may be stored on any type of attached array of writable storage device media such as video tape, optical, DVD, magnetic tape, bubble memory, electronic random access memory, micro-electromechanical and any other similar media adapted to store information, including data and parity information. However, as illustratively described herein, the information is stored on disks (e.g., associated with storage pod

145). The storage adapter comprises a plurality of ports having input/output (I/O) interface circuitry that couples to the disks over an I/O interconnect arrangement, such as a conventional high-performance, FC link topology.

[0068] Storage of information on each disk array may be implemented as one or more storage "volumes" that comprise a collection of physical storage disks or cloud volumes cooperating to define an overall logical arrangement of volume block number (VBN) space on the volume(s). Each logical volume is generally, although not necessarily, associated with its own file system. The disks within a logical volume/file system are typically organized as one or more groups, wherein each group may be operated as a Redundant Array of Independent (or Inexpensive) Disks (RAID). Most RAID implementations, such as a RAID-4 level implementation, enhance the reliability/integrity of data storage through the redundant writing of data "stripes" across a given number of physical disks in the RAID group, and the appropriate storing of parity information with respect to the striped data. An illustrative example of a RAID implementation is a RAID-4 level implementation, although it should be understood that other types and levels of RAID implementations may be used in accordance with the inventive principles described herein.

[0069] While in the context of the present example, the node may be a physical host, it is to be appreciated the node may be implemented in virtual form. For example, a storage system may be run (e.g., on a VM or as a containerized instance, as the case may be) within a public cloud provider. As such, a cluster representing a distributed storage system may be comprised of multiple physical nodes (e.g., node **200**) or multiple virtual nodes (virtual storage systems).

Example Storage Operating System

[0070] To facilitate access to the disks (e.g., disks within one or more disk arrays of a storage pod, such as storage pod **145** of FIG. **1**), a storage operating system (e.g., storage operating system **300**, which may be analogous to storage operating system **210**) may implement a write-anywhere file system that cooperates with one or more virtualization modules to "virtualize" the storage space provided by disks. The file system logically organizes the information as a hierarchical structure of named directories and files on the disks. Each "on-disk" file may be implemented as set of disk blocks configured to store information, such as data, whereas the directory may be implemented as a specially formatted file in which names and links to other files and directories are stored. The virtualization module(s) allow the file system to further logically organize information as a hierarchical structure of blocks on the disks that are exported as named logical unit numbers (LUNs).

[0071] Illustratively, the storage operating system may be the Data ONTAP operating system available from NetApp, Inc., San Jose, Calif. that implements the WAFL® file system. However, it is expressly contemplated that any appropriate storage operating system may be enhanced for use in accordance with the inventive principles described herein. As such, where the term "WAFL" is employed, it should be taken broadly to refer to any file system that is otherwise adaptable to the teachings of this disclosure.

[0072] FIG. **3** is a block diagram illustrating a storage operating system **300** in accordance with an embodiment of the present disclosure. In the context of the present example, the storage operating system **300** is shown including a series of software layers organized to form an integrated network protocol stack or, more generally, a multi-protocol engine **325** that provides data paths for clients to access information stored on the node using block and file access protocols. The multi-protocol engine includes a media access layer **312** of network drivers (e.g., gigabit Ethernet drivers) that interfaces to network protocol layers, such as the IP layer **314** and its supporting transport mechanisms, the TCP layer **316** and the User Datagram Protocol (UDP) layer **315**. A file system protocol layer provides multi-protocol file access and, to that end, includes support for the Direct Access File System (DAFS) protocol **318**, the NFS protocol **320**, the CIFS protocol **322** and the Hypertext Transfer Protocol (HTTP) protocol **324**. A VI layer **326** implements the VI architecture to provide direct access transport (DAT) capabilities, such as RDMA, as required by the DAFS

protocol **318**. An iSCSI driver layer **328** provides block protocol access over the TCP/IP network protocol layers, while a FC driver layer **330** receives and transmits block access requests and responses to and from the node. The FC and iSCSI drivers provide FC-specific and iSCSI-specific access control to the blocks and, thus, manage exports of LUNs to either iSCSI or FCP or, alternatively, to both iSCSI and FCP when accessing the blocks on the node (e.g., node **200**).

[0073] In addition, the storage operating system may include a series of software layers organized to form a storage server **365** that provides data paths for accessing information stored on the disks (e.g., disks **130**) of the node. To that end, the storage server **365** includes a file system module **360** in cooperating relation with a remote access module **370**, a RAID system module **380** and a disk driver system module **390**. The RAID system **380** manages the storage and retrieval of information to and from the volumes/disks in accordance with I/O operations, while the disk driver system **390** implements a disk access protocol such as, e.g., the SCSI protocol.

[0074] The file system **360** may implement a virtualization system of the storage operating system **300** through the interaction with one or more virtualization modules illustratively embodied as, for example, a virtual disk (vdisk) module (not shown) and a SCSI target module **335**. The SCSI target module **335** is generally disposed between the FC and iSCSI drivers **328**, **330** and the file system **360** to provide a translation layer of the virtualization system between the block (LUN) space and the file system space, where LUNs are represented as blocks.

[0075] The file system **360** is illustratively a message-based system that provides logical volume management capabilities for use in access to the information stored on the storage devices, such as disks. That is, in addition to providing file system semantics, the file system **360** provides functions normally associated with a volume manager. These functions include (i) aggregation of the disks, (ii) aggregation of storage bandwidth of the disks, and (iii) reliability guarantees, such as mirroring and/or parity (RAID). The file system **360** illustratively implements an exemplary a file system having an on-disk format representation that is block-based using, e.g., 4 kilobyte (KB) blocks and using index nodes (“inodes”) to identify files and file attributes (such as creation time, access permissions, size and block location). The file system may use files to store metadata describing the layout of its file system; these metadata files may include, among others, an inode file. A file handle (e.g., an identifier that includes an inode number) may be used to retrieve an inode from disk.

[0076] Broadly stated, all inodes of the write-anywhere file system are organized into the inode file. A file system (fs) info block specifies the layout of information in the file system and includes an inode of a file that includes all other inodes of the file system. Each logical volume (file system) has an fsinfo block that is preferably stored at a fixed location within, e.g., a RAID group. The inode of the inode file may directly reference (point to) data blocks of the inode file or may reference indirect blocks of the inode file that, in turn, reference data blocks of the inode file. Within each data block of the inode file are embedded inodes, each of which may reference indirect blocks that, in turn, reference data blocks of a file.

[0077] Operationally, a request from a client (e.g., client **180**) is forwarded as a packet over a computer network (e.g., computer network **140**) and onto a node (e.g., node **200**) where it is received at a network adapter (e.g., network adaptor **225**). A network driver (of layer **312** or layer **330**) processes the packet and, if appropriate, passes it on to a network protocol and file access layer for additional processing prior to forwarding to the write-anywhere file system **360**. Here, the file system generates operations to load (retrieve) the requested data from disk **130** if it is not resident “in core”, i.e., in memory **224**. If the information is not in memory, the file system **360** indexes into the inode file using the inode number to access an appropriate entry and retrieve a logical VBN. The file system then passes a message structure including the logical VBN to the RAID system **380**; the logical VBN is mapped to a disk identifier and disk block number (disk,dbn) and sent to an appropriate driver (e.g., SCSI) of the disk driver system **390**. The disk driver accesses the dbn from the specified disk **130** and loads the requested data block(s) in memory for

processing by the node. Upon completion of the request, the node (and operating system) returns a reply to the client **180** over the network **140**.

[0078] The remote access module **370** is operatively interfaced between the file system module **360** and the RAID system module **380**. Remote access module **370** is illustratively configured as part of the file system to implement the functionality to determine whether a newly created data container, such as a subdirectory, should be stored locally or remotely. Alternatively, the remote access module **370** may be separate from the file system. As such, the description of the remote access module being part of the file system should be taken as exemplary only. Further, the remote access module **370** determines which remote flexible volume should store a new subdirectory if a determination is made that the subdirectory is to be stored remotely. More generally, the remote access module **370** implements the heuristics algorithms used for the adaptive data placement. However, it should be noted that the use of a remote access module should be taken as illustrative. In alternative aspects, the functionality may be integrated into the file system or other module of the storage operating system. As such, the description of the remote access module **370** performing certain functions should be taken as exemplary only.

[0079] It should be noted that the software "path" through the storage operating system layers described above needed to perform data storage access for the client request received at the node may alternatively be implemented in hardware. That is, a storage access request data path may be implemented as logic circuitry embodied within a field programmable gate array (FPGA) or an application specific integrated circuit (ASIC). This type of hardware implementation increases the performance of the storage service provided by node **200** in response to a request issued by client **180**. Alternatively, the processing elements of adapters **225**, **228** may be configured to offload some or all of the packet processing and storage access operations, respectively, from processor **222**, to thereby increase the performance of the storage service provided by the node. It is expressly contemplated that the various processes, architectures and procedures described herein can be implemented in hardware, firmware or software.

[0080] As used herein, the term "storage operating system" generally refers to the computer-executable code operable on a computer to perform a storage function that manages data access and may, in the case of a node (e.g., node **200**), implement data access semantics of a general purpose operating system. The storage operating system can also be implemented as a microkernel, an application program operating over a general-purpose operating system, such as UNIX or Windows NT, or as a general-purpose operating system with configurable functionality, which is configured for storage applications as described herein.

[0081] In addition, it will be understood to those skilled in the art that aspects of the disclosure described herein may apply to any type of special-purpose (e.g., file server, filer or storage serving appliance) or general-purpose computer, including a standalone computer or portion thereof, embodied as or including a storage system. Moreover, the teachings contained herein can be adapted to a variety of storage system architectures including, but not limited to, a network-attached storage environment, a storage area network and disk assembly directly attached to a client or host computer. The term "storage system" should therefore be taken broadly to include such arrangements in addition to any subsystems configured to perform a storage function and associated with other equipment or systems. It should be noted that while this description is written in terms of a write anywhere file system, the teachings of the subject matter may be utilized with any suitable file system, including a write in place file system.

Example Cluster Fabric (CF) Protocol

[0082] Illustratively, the storage server **365** is embodied as disk element (or disk blade **350**, which may be analogous to disk element **150***a* or **150***b*) of the storage operating system **300** to service one or more volumes of array **160**. In addition, the multi-protocol engine **325** is embodied as network element (or network blade **310**, which may be analogous to network element **120***a* or **120***b*) to (*i*) perform protocol termination with respect to a client issuing incoming data access request packets

over the network (e.g., network **140**), as well as (ii) redirect those data access requests to any storage server **365** of the cluster (e.g., cluster **100**). Moreover, the network element **310** and disk element **350** cooperate to provide a highly scalable, distributed storage system architecture of the cluster. To that end, each module may include a cluster fabric (CF) interface module (e.g., CF interface **340***a* and **340***b*) adapted to implement intra-cluster communication among the nodes (e.g., node **110***a* and **110***b*). In the context of a distributed storage architecture as described below with reference to FIG. **5** in which node-level aggregates are employed, the CF protocol facilitates, among other things, internode communications relating to data access requests. It is to be appreciated such internode communications relating to data access requests are not needed in the context of a distributed storage architecture as described below with reference to FIG. **6**A in which each node of a cluster has visibility and access to the entirety of a global PVBN space of a storage pod (via their respective DEFSs). However, in various embodiments, some limited amount of internode communications, for example, relating to storage space reporting (or simply space reporting) and storage space requests (e.g., requests for donations of AAs) continue to be useful. As described further below, such internode communications may make use of the CF protocol or other forms of internode communications, including message passing via on-wire communications and/or the use of one or more persistent message queues (or on-disk message queues), which may make use of the fact that all nodes can read from all disk of a storage pod. For example, a persistent message queue may be maintained at the node and/or DEFS-level of granularity in which each node and/or DEFS has a message queue to which others can post messages destined for the node or DEFS (as the case may be). In one embodiment, each DEFS has an associated inbound queue on which it receives messages sent by another DEFS in the cluster and an associated outbound queue on which it posts messages intended for delivery to another DEFS in the cluster

[0083] The protocol layers, e.g., the NFS/CIFS layers and the iSCSI/IFC layers, of the network element **310** may function as protocol servers that translate file-based and block based data access requests from clients into CF protocol messages used for communication with the disk element **350**. That is, the network element servers may convert the incoming data access requests into file system primitive operations (commands) that are embedded within CF messages by the CF interface module **340** for transmission to the disk elements of the cluster.

[0084] Further, in an illustrative aspect of the disclosure, the network element and disk element are implemented as separately scheduled processes of storage operating system **300**; however, in an alternate aspect, the modules may be implemented as pieces of code within a single operating system process. Communication between a network element and disk element may thus illustratively be effected through the use of message passing between the modules although, in the case of remote communication between a network element and disk element of different nodes, such message passing occurs over a cluster switching fabric (e.g., cluster switching fabric **151**). A known message-passing mechanism provided by the storage operating system to transfer information between modules (processes) is the Inter Process Communication (IPC) mechanism. The protocol used with the IPC mechanism is illustratively a generic file and/or block-based "agnostic" CF protocol that comprises a collection of methods/functions constituting a CF application programming interface (API). Examples of such an agnostic protocol are the SpinFS and SpinNP protocols available from NetApp, Inc.

[0085] The CF interface module **340** implements the CF protocol for communicating file system commands among the nodes or modules of cluster. Communication may be illustratively effected by the disk element exposing the CF API to which a network element (or another disk element) issues calls. To that end, the CF interface module **340** may be organized as a CF encoder and CF decoder. The CF encoder of, e.g., CF interface **340***a* on network element **310** encapsulates a CF message as (i) a local procedure call (LPC) when communicating a file system command to a disk element **350** residing on the same node **200** or (ii) a remote procedure call (RPC) when communicating the command to a disk element residing on a remote node of the cluster **100**. In

either case, the CF decoder of CF interface **340***b* on disk element **350** de-encapsulates the CF message and processes the file system command.

[0086] Illustratively, the remote access module **370** may utilize CF messages to communicate with remote nodes to collect information relating to remote flexible volumes. A CF message is used for RPC communication over the switching fabric between remote modules of the cluster; however, it should be understood that the term "CF message" may be used generally to refer to LPC and RPC communication between modules of the cluster. The CF message includes a media access layer, an IP layer, a UDP layer, a reliable connection (RC) layer and a CF protocol layer. The CF protocol is a generic file system protocol that may convey file system commands related to operations contained within client requests to access data containers stored on the cluster; the CF protocol layer is that portion of a message that carries the file system commands. Illustratively, the CF protocol is datagram based and, as such, involves transmission of messages or "envelopes" in a reliable manner from a source (e.g., a network element **310**) to a destination (e.g., a disk element **350**). The RC layer implements a reliable transport protocol that is adapted to process such envelopes in accordance with a connectionless protocol, such as UDP.

Example File System Layout

[0087] In one embodiment, a data container is represented in the write-anywhere file system as an inode data structure adapted for storage on the disks of a storage pod (e.g., storage pod **145**). In such an embodiment, an inode includes a metadata section and a data section. The information stored in the metadata section of each inode describes the data container (e.g., a file, a snapshot, etc.) and, as such, includes the type (e.g., regular, directory, vdisk) of file, its size, time stamps (e.g., access and/or modification time) and ownership (e.g., user identifier (UID) and group ID (GID), of the file, and a generation number. The contents of the data section of each inode may be interpreted differently depending upon the type of file (inode) defined within the type field. For example, the data section of a directory inode includes metadata controlled by the file system, whereas the data section of a regular inode includes file system data. In this latter case, the data section includes a representation of the data associated with the file.

[0088] Specifically, the data section of a regular on-disk inode may include file system data or pointers, the latter referencing 4 KB data blocks on disk used to store the file system data. Each pointer is preferably a logical VBN to facilitate efficiency among the file system and the RAID system when accessing the data on disks. Given the restricted size (e.g., 128 bytes) of the inode, file system data having a size that is less than or equal to 64 bytes is represented, in its entirety, within the data section of that inode. However, if the length of the contents of the data container exceeds 64 bytes but less than or equal to 64 KB, then the data section of the inode (e.g., a first level inode) comprises up to 16 pointers, each of which references a 4 KB block of data on the disk.

[0089] Moreover, if the size of the data is greater than 64 KB but less than or equal to 64 megabytes (MB), then each pointer in the data section of the inode (e.g., a second level inode) references an indirect block (e.g., a first level L1 block) that contains 224 pointers, each of which references a 4 KB data block on disk. For file system data having a size greater than 64 MB, each pointer in the data section of the inode (e.g., a third level L3 inode) references a double-indirect block (e.g., a second level L2 block) that contains 224 pointers, each referencing an indirect (e.g., a first level L1) block. The indirect block, in turn, which contains 224 pointers, each of which references a 4 kB data block on disk. When accessing a file, each block of the file may be loaded from disk into memory (e.g., memory **224**). In other embodiments, higher levels are also possible that may be used to handle larger data container sizes.

[0090] When an on-disk inode (or block) is loaded from disk into memory, its corresponding in-core structure embeds the on-disk structure. The in-core structure is a block of memory that stores the on-disk structure plus additional information needed to manage data in the memory (but not on disk). The additional information may include, e.g., a "dirty" bit. After data in the inode (or block)

is updated/modified as instructed by, e.g., a write operation, the modified data is marked "dirty" using the dirty bit so that the inode (block) can be subsequently "flushed" (stored) to disk.
[0091] According to one embodiment, a file in a file system comprises a buffer tree that provides an internal representation of blocks for a file loaded into memory and maintained by the write-anywhere file system **360**. A root (top-level) buffer, such as the data section embedded in an inode, references indirect (e.g., level 1) blocks. In other embodiments, there may be additional levels of indirect blocks (e.g., level 2, level 3) depending upon the size of the file. The indirect blocks (e.g., and inode) includes pointers that ultimately reference data blocks used to store the actual data of the file. That is, the data of file are contained in data blocks and the locations of these blocks are stored in the indirect blocks of the file. Each level 1 indirect block may include pointers to as many as 224 data blocks. According to the "write anywhere" nature of the file system, these blocks may be located anywhere on the disks.
[0092] In one embodiment, a file system layout is provided that apportions an underlying physical volume into one or more virtual volumes (or flexible volumes) of a storage system, such as node **200**. In such an embodiment, the underlying physical volume is an aggregate comprising one or more groups of disks, such as RAID groups, of the node. The aggregate has its own physical volume block number (PVBN) space and maintains metadata, such as block allocation structures, within that PVBN space. Each flexible volume has its own virtual volume block number (VVBN) space and maintains metadata, such as block allocation structures, within that VVBN space. Each flexible volume is a file system that is associated with a container file; the container file is a file in the aggregate that contains all blocks used by the flexible volume. Moreover, each flexible volume comprises data blocks and indirect blocks that contain block pointers that point at either other indirect blocks or data blocks.
[0093] In a further embodiment, PVBNs are used as block pointers within buffer trees of files stored in a flexible volume. This "hybrid" flexible volume example involves the insertion of only the PVBN in the parent indirect block (e.g., inode or indirect block). On a read path of a logical volume, a "logical" volume (vol) info block has one or more pointers that reference one or more fsinfo blocks, each of which, in turn, points to an inode file and its corresponding inode buffer tree. The read path on a flexible volume is generally the same, following PVBNs (instead of VVBNs) to find appropriate locations of blocks; in this context, the read path (and corresponding read performance) of a flexible volume is substantially similar to that of a physical volume. Translation from PVBN-to-disk,dbn occurs at the file system/RAID system boundary of the storage operating system **300**.
[0094] In a dual VBN hybrid flexible volume example, both a PVBN and its corresponding VVBN are inserted in the parent indirect blocks in the buffer tree of a file. That is, the PVBN and VVBN are stored as a pair for each block pointer in most buffer tree structures that have pointers to other blocks, e.g., level 1 (L1) indirect blocks, inode file level 0 (L0) blocks.
[0095] A root (top-level) buffer, such as the data section embedded in an inode, references indirect (e.g., level 1) blocks. Note that there may be additional levels of indirect blocks (e.g., level 2, level 3) depending upon the size of the file. The indirect blocks (and inode) include PVBN/VVBN pointer pair structures that ultimately reference data blocks used to store the actual data of the file. The PVBNs reference locations on disks of the aggregate, whereas the VVBNs reference locations within files of the flexible volume. The use of PVBNs as block pointers in the indirect blocks provides efficiencies in the read paths, while the use of VVBN block pointers provides efficient access to required metadata. That is, when freeing a block of a file, the parent indirect block in the file contains readily available VVBN block pointers, which avoids the latency associated with accessing an owner map to perform PVBN-to-VVBN translations; yet, on the read path, the PVBN is available.
Example Hierarchical Inode Tree
[0096] FIG. **4** is a block diagram illustrating a tree of blocks **400** representing a simplified view of

an example a file system layout in accordance with an embodiment of the present disclosure. In one embodiment, the data storage system nodes (e.g., data storage systems **110***a-b*) make use of a write anywhere file system (e.g., the WAFL® file system). The write anywhere file system may represent a UNIX® compatible file system that is optimized for network file access. In the context of the present example, the write anywhere file system is a block-based file system that represents file system data (e.g., a block map file and an inode map file), metadata files, and data containers (e.g., volumes, subdirectories, and regular files) in a tree of blocks (e.g., tree of blocks **400**). Keeping metadata in files allows the file system to write metadata blocks anywhere on disk and makes it easier to increase the size of the file system on the fly.

[0097] In this simplified example, the tree of blocks **400** has a root inode **410**, which describes an inode map file (not shown), made up of inode file indirect blocks **420** and inode file data blocks **430**. In this example, the file system uses inodes (e.g., inode file data blocks **430**) to describe data containers representing files (e.g., file **460**). In one embodiment, each inode contains 16 block pointers to indicate which blocks (e.g., of 4 KB) belong to a given data container (e.g., a file). Inodes for data containers smaller than 64 KB may use the 156 block pointers to point to file data blocks or simply data blocks (e.g., regular file data blocks, which may also be referred to herein as L0 blocks **450**). Inodes for files smaller than 64 MB may point to indirect blocks (e.g., regular file indirect blocks, which may also be referred to herein as L1 blocks **440**), which point to actual file data. Inodes for larger files or data containers may point to doubly indirect blocks. For very small files, data may be stored in the inode itself in place of the block pointers.

[0098] In the context of the present example, an inode **435** is shown including a buffer tree identifier (i.e., bufftree ID **432**) and pointers **431***a-n* (e.g., PVBNs) to indirect (or L1) blocks that in turn point to data (or L0) blocks containing the file data. An example of a data block format including data, a checksum, and context data is described below with reference to FIG. **8**A. The bufftree ID **432** may represent a cluster-wide, unique volume ID assigned to the volume with which the file **460** is associated and may be used to facilitate performance of context checking during processing of internal (e.g., those initiated by workflows or subsystems of the storage system) read requests and/or external (e.g., those initiated by clients of the storage system) read requests to avoid returning stale data to the requestor. In various embodiments described herein, given the fact that files may be moved from one DEFS to another within the cluster (e.g., as a result of volume movement), it is desirable for the bufftree ID to be unique across the cluster to avoid bufftree ID collisions. Non-limiting examples of bufftree ID formats that may be used for client data files and metafiles (e.g., containing metadata used by the file system) are described further below with reference to FIGS. **8**B and **8**C, respectively.

[0099] As will be appreciated by those skilled in the art given the above-described file system layout, yet another advantage of DEFSs are their ability to facilitate storage space balancing and/or load balancing. This comes from the fact that the entire global PVBN space of a storage pod is visible to all DEFSs of the cluster and therefore any given DEFS can get access to an entire file by copying the top-most PVBN from the inode on another tree.

Example of a Distributed Storage System Architecture with Storage Silos

[0100] FIG. **5** is a block diagram illustrating a distributed storage system architecture **500** in which the entirety of a given disk and a given RAID group are owned by an aggregate and the aggregate file system is only visible from one node, thereby resulting in silos of storage space. In the context of FIG. **5**, node **510***a* and node **510***b* may represent a two-node cluster in which the nodes are high-availability (HA) partners. For example, one node may represent a primary node and the other may represent a secondary node in which pairwise disk connectively supports a pairwise failover model. As shown, each node includes respective active maps (e.g., active map **541***a* and active map **541***b*) and a sets of disks (in this case, ten disks) they can talk to. The nodes may partition the disks among themselves as aggregates (e.g., data aggregate **520***a* and data aggregate **520***b*) and at steady state both nodes will work on their own subset of disks representing a one or more RAID groups

(in this case, four data disks and one parity disk, forming a single RAID group). A RAID layer or subsystem (not shown) of a storage operating system (not shown) of each node may present respective separate and independent PVBN spaces (e.g., PVBN space **540***a* and PVBN space **540***b*) to a file system layer (not shown) of the node.

[0101] In this example, therefore, data aggregate **520***a* has visibility only to a first PVBN space (e.g., PVBN space **540***a*) and data aggregate **520***b* has visibility only to a second PVBN space (e.g., PVBN space **540***b*). When data is stored to volume **530***a* or **530***b*, it is striped across the subset of disks that are part of data aggregate **520***a*; and when data is stored to volume **530***c* or **530***d*, it is are striped across the subset of disks that are part of data aggregate **520***b*. Active map **541***a* is a data structure (e.g., a bit map with one bit per PVBN) that that identifies the PVBNs within PVBN space **540***a* that are in use by data aggregate **520***a*. Similarly, active map **541***b* is a data structure (e.g., a bit map with one bit per PVBN) that that identifies the PVBNs within PVBN space **540***b* that are in use by data aggregate **520***b*.

[0102] As can be seen, for any given disk, the entire disk is owned by a particular aggregate and the aggregate file system is only visible from one node. Similarly, for any given RAID group, the available storage space of the entire RAID group is useable only by a single node. There are various other disadvantages to the architecture shown in FIG. **5**. For example, moving a volume from one aggregate to another requires copying of data (e.g., reading all the blocks used by the volume and writing them to the new location), with an elaborate handover sequence between the aggregates involved. Additionally, there are scenarios in which one data aggregate may run out of storage space while the other still has plentiful free storage space, resulting in ineffective usage of the storage space provided by the disks. While the size of the PVBN space of an aggregate may be increased, doing so typically requires an administrative user to monitor the storage space on each node-level aggregate and add one or more disks and/or RAID groups to the aggregate. As described further below with reference to FIG. **6**A, with DEFSs storage space is added to a common pool of storage referred to herein as a “storage pod” and space is available for consumption by any DEFS in the cluster, thereby making space management much simpler and facilitating the automatic balancing of storage space without administrator involvement.

Example Distributed System Architecture Providing Disaggregated Storage

[0103] Before getting into the details of a particular example, various properties, constructs, and principles relating to the use and implementation of DEFSs will now be discussed. As noted above, it is desirable to make the global PVBN space of the entire storage pool available on each DEFS of a data pod, which may include one or more clusters. This feature facilitates the performance of, among other things, instant copy-free moves of volumes from one DEFS to another, for example, in connection with performing load balancing. Creating clones on remote nodes for load balancing is yet another benefit. With a global PVBN space, support for global data deduplication can also be supported rather than deduplication being limited to node-level aggregates.

[0104] It is also beneficial, in terms of performance, to avoid the use of access control mechanism, such as locks, to coordinate write accesses and write allocation among nodes generally and DEFSs specifically. Such access control mechanisms may be eliminated by specifying, at a per-DEFS level, those portions of the disaggregated storage of the storage pod to which a given DEFS has exclusive write access. For example, as described further below, a DEFS may be limited to use of only the AAs associated with (assigned to or owned by) the DEFS for performing write allocation and write accesses during a CP. Advantageously, given the visibility into the entire global PVBN space, reads can be performed by any DEFS of the cluster from all the PVBNs in the storage pod.

[0105] Each DEFS of a given cluster (or data pod, as the case may be) may start at its own super block. As shown and described with reference to FIG. **6**A, a predefined AA (e.g., the first AA) in storage pod may be dedicated for super blocks. In one embodiment, a set of RAID stripes within the predefined super block AA (e.g., the first AA of the storage pod) may be dedicated for super blocks. In this predefined super block AA, ownership may be specified at the granularity of a single

RAID stripe instead of at the AA granularity of multiple RAID stripes representing one or more GBs (e.g., between approximately 1 GB and 10 GB) of storage space. The location of a super block of a given DEFS can be mathematically derived using an identifier (a DEFS ID) associated with the given DEFS. Since the RAID stripe is already reserved for a super block, it can be replicated on N disks.

[0106] Each DEFS has AAs associated with it, which may be thought of conceptually as the DEFS owning those AAs. In one embodiment, AAs may be tracked within an AA map and persisted within the DEFS filesystem. An AA map may include the DEFS ID in an AA index. While AA ownership information regarding other DEFSs in the cluster may be cached in the AA map of a given DEFS, which may be useful during the PVBN free path, for example, to facilitate freeing of PVBNs of an AA not owned by the given DEFS (which may arise in situations in which partial AAs are donated from one DEFS to another), the authoritative source information regarding the AAs owned by a given DEFS may be presumed to be in the AA map of the given DEFS.

[0107] In support of avoiding storage silos and supporting the more fluid use of disk space across all nodes of a cluster, DEFSs may be allowed to donate partially or completely free AAs to other DEFSs.

[0108] Each DEFS may have its own label information kept in the file system. The label information may be kept in the super block or another well-known location outside of the file system.

[0109] In various examples, there can be multiple DEFSs on a RAID tree. That is, there may be a many-to-one association between DEFSs and a RAID tree, in which each DEFS may have a reference on the RAID tree. The RAID tree can still have multiple RAID groups. In various examples described herein, it is assumed the PVBN space provided by the RAID tree is continuous.

[0110] It may be helpful to have a root DEFS and a data DEFS that are transparent to other subsystems. These DEFSs may be useful for storing information that might be needed before the file system is brought online. Examples of such information may include controller (node) failover (CFO) and storage failover (SFO) properties/policies. HA is one example of where it might be helpful to bring up a controller (node) failover root DEFS first before giving back the storage failover data DEFSs. HA coordination of bringing down a given DEFS on takeover/giveback may be handled by the file system (e.g., the WAFL® file system) since the RAID tree would be up until the node is shutdown.

[0111] DEFS data structures (e.g., DEFS bit maps at the PVBN level, such as active maps and reference count (refcount) maps) may be sparse. That is, they may represent the entire global PVBN space, but only include valid truth values for PVBNs of AAs that are owned by the particular DEFS with which they are associated. When validation of these bit maps is performed by or on behalf of a particular DEFS, the bits should be validated only for the AA areas owned by the particular DEFS. When using such sparce data structures, to get the complete picture of the PVBN space, the data structures in all of the nodes should be taken into consideration. While various DEFS data structures may be discussed herein as if they were separate metafiles, it is to be appreciated, given the visibility by each node into the entire global PVBN space, one or more of such DEFS data structures may be represented as cluster-wide metafiles. Such a cluster-wide metafile may be persisted in a private inode space that is not accessible to end users and the relevant portions for a particular DEFS may be located based on the DEFS ID of the particular DEFS, for example, which may be associated with the appropriate inode (e.g., an L0 block). Similarly, the entirety of such a cluster-wide metafile may be accessible based on a cluster ID, for example, which may be associated with a higher-level inode in the hierarchy (e.g., an L1 block). In any event, each node should generally have all the information it needs to work independently until and unless it runs out of storage space or meets a predetermined or configurable threshold of a storage space metric (e.g., a free space metric or a used space metric), for example, relative to the other nodes of the cluster. At that point, as described further below, as part of a space monitoring

and/or a space balancing process, the node may request a portion of AAs of DEFSs owned by one or more of such other nodes be donated so as to increase the useable storage space of one or more DEFSs of the node at issue.

[0112] FIG. **6**A is a block diagram illustrating a distributed storage system architecture **600** that provides disaggregated storage in accordance with an embodiment of the present disclosure. Various architectural advantages of the proposed distributed storage system architecture and mechanisms for providing and making use of disaggregated storage include, but are not limited to, the ability to perform automatic space balancing among DEFSs, perform elastic node growth and shrinkage for a cluster, perform elastic storage growth of the storage pod, perform zero-copy file and volume move (migration), perform distributed RAID rebuild, achieve HA cost reduction using volume rehosting, create remote clones, and perform global data deduplication.

[0113] In the context of the present example, the nodes (e.g., node **610***a* and **610***b*) of a cluster, which may represent a data pod or include multiple data pods, each include respective data dynamically extensible file systems (DEFSs) (e.g., data DEFS **620***a* and data DEFS **620***b*) and respective log DEFSs (e.g., log DEFS **625***a* and log DEFS **625***b*). In general, data DEFSs may be used for persisting data on behalf of clients (e.g., client **180**), whereas log DEFSs may be used to maintain an operation log or journal of certain storage operations within the journaling storage media that have been performed since the last CP.

[0114] It should be noted that while for simplicity only two nodes, which may be configured as part of an HA pair for fault tolerance and nondisruptive operations, are shown in the illustrative cluster depicted in FIG. **6**A, there may be one or more additional nodes in a given cluster. For example, there may be multiple HA pairs within a cluster (or a data pod of the cluster, which may represent a mechanism to limit the fault domain). As such, the description of this two-node cluster should be taken as illustrative only. Furthermore, while in some examples HA may be achieved by defining pairs of nodes within a cluster as HA partners (e.g., with one node designated as the primary node and the other designated as the secondary), in alternative examples any other node within a cluster may be allowed to step in after a failure of a given node without defining HA pairs.

[0115] As discussed above, one or more volumes (e.g., volumes **630***a-m* and volumes **630***n-x*) or LUNs (not shown) may be created by or on behalf of customers for hosting/storing their enterprise application data within respective DEFSs (e.g., data DEFSs **620***a* and **620***b*).

[0116] While additional data structures may be employed, in this example, each DEFS is shown being associated with respective AA maps (indexed by AA ID) and active maps (indexed by PVBN). For example, log DEFS **625***a* may utilize AA map **627***a* to track those of the AAs within a global PVBN space **640** of storage pod **645** (which may be analogous to storage pod **145**) that are owned by log DEFS **625***a* and may utilize active map **626***a* to track at a PVBN level of granularity which of the PVBNs of its AAs are in use; log DEFS **625***b* may utilize AA map **627***b* to track those of the AAs within the global PVBN space **640** that are owned by log DEFS **625***b* and may utilize active map **626***b* to track at a PVBN level of granularity which of the PVBNs of its AAs are in use; data DEFS **620***a* may utilize AA map **622***a* to track those of the AAs within the global PVBN space **640** that are owned by data DEFS **620***a* and may utilize active map **621***a* to track at a PVBN level of granularity which of the PVBNs of its AAs are in use; and data DEFS **620***b* may utilize AA map **622***b* to track those of the AAs within the global PVBN space **640** that are owned by data DEFS **620***b* and may utilize active map **621***b* to track at a PVBN level of granularity which of the PVBNs of its AAs are in use.

[0117] In this example, each DEFS of a given node has visibility and accessibility into the entire global PVBN address space **640** and any AA (except for a predefined super block AA **642**) within the global PVBN address space **640** may be assigned to any DEFS within the cluster. By extension, each node has visibility and accessibility into the entire global PVBN address space **640** via its DEFSs. As noted above, the respective AA maps of the DEFSs define which PVBNs to which the DEFSs have exclusive write access. AAs within the global PVBN space **640** shaded in light gray,

such as AA **641***a*, can only be written to by node **610***a* as a result of their ownership by or assignment to data DEFS **620***a*. Similarly AAs within the global PVBN space **640** shaded in dark gray, such as AA **641***b*, can only be written to by node **610***b* as a result of their ownership by or assignment to data DEFS **620***b*.

[0118] Returning to super block **642**, it is part of a super block AA (or super AA). In the context of FIG. **6**A, the super AA is the first AA of the storage pod **645**. The super AA is not assigned to any DEFS (as indicated by its lack of shading). The super AA may have an array of DEFS areas which are dedicated to each DEFS and can be indexed by a DEFS ID. The DEFS ID may start at index **1** and in the context of the present example includes four super block and four DEFS label blocks. The DEFS label can act as a RAID label for the DEFS and can be written out of a CP and can store information that needs to be kept outside of the file system. In a pairwise HA configuration, two super blocks and two DEFS label blocks may be used by the hosting node and the other two may be used by the partner node on takeover. Each of these special blocks may have their own separate stripes.

[0119] In the context of the present example, it is assumed after establishment of the disaggregated storage within the storage pod **645** and after the original assignment of ownership of AAs to data DEFS **620***a* and data DEFS **620***b*, some AAs have been transferred from data DEFS **620***a* to data DEFS **620***b* and/or some AAs have been transferred from data DEFS **620***b* to data DEFS **620***a*. As such, the different shades of grayscale of entries within the AA maps are intended to represent potential caching that may be performed regarding ownership of AAs owned by other DEFSs in the cluster. For example, assuming ownership of a partial AA has been transferred from data DEFS **620***a* to data DEFS **620***b* as part of an ownership change performed in support of space balancing, when data DEFS **620***a* would like to free a given PVBN (e.g., when the given PVBN is no longer referenced by data DEFS **620***a* a result of data deletion or otherwise), data DEFS **620***a* should send a request to free the PVBN to the new owner (in this case, data DEFS **620***b*). This is due to the fact that in various embodiments, only the current owner of a particular AA is allowed to perform any modify operations on the particular AA. While not necessary for understanding context checks, further explanation regarding space balancing and AA ownership change is provided in co-pending U.S. patent application Ser. No. 19/068,324 (previously incorporated by reference above) and co-pending U.S. patent application Ser. No. 18/595,785, filed on Mar. 5, 1924, which is hereby incorporated by reference in its entirety for all purposes.

[0120] Those skilled in the art will appreciate disaggregation of the storage space as discussed herein can be leveraged for cost-effective scaling of infrastructure. For example, the disaggregated storage allows more applications to share the same underlying storage infrastructure. Given that each DEFS represents an independent file system, the use of multiple of such DEFSs combine to create a cluster-wide distributed file system since all of the DEFSs within a cluster share a global PVBN space (e.g., global PVBN space **640**). This provides the unique ability to independently scale each independent DEFS as well as enables fault isolation and repair in a manner different from existing distributed file systems.

[0121] Additional aspects of FIG. **6**A will now be described in connection with a discussion of FIG. **6**B, which represents a high-level flow diagram illustrating operations for establishing disaggregated storage within a storage pod (e.g., storage pod **645**). The processing described with reference to FIG. **6**B, may be performed by a combination of a file system (e.g., file system **360**) and a RAID system (e.g., RAID system **380**), for example, during or after an initial boot up.

[0122] At block **661**, the storage pod is created based on a set of disks made available for use by the cluster. For example, job may be executed by a management plane of the cluster to create the storage pod and assign the disks to the cluster. Depending on the particular implementation and the deployment environment (e.g., on-prem versus cloud), the disks may be associated with of one or more disk arrays or one or more storage shelves or persistent storage in the form of cloud volumes provided by a cloud provider from a pool of storage devices within a cloud environment. For

simplicity, cloud volumes may also be referred to herein as “disks.” The disks may be HDDs or SSDs.

[0123] At block **662**, the storage space of the set of disks may be divided or partitioned into uniform-sized AAs. The set of disks may be grouped to form multiple RAID groups (e.g., RAID group **650***a* and **650***b*) depending on the RAID level (e.g., RAID 4, RAID 5, or other). Multiple RAID stripes may then be grouped to form individual AAs. As noted above, an AA (e.g., AA **641***a* or AA **641***b*) may be a large chunk representing one or more GB of storage space and preferably accommodates multiple SSD erase blocks work of data. In one embodiment, the size of the AAs is tuned for the particular file system. The size of the AAs may also take into consideration a desire to reduce the need for performing space balancing so as to minimize the need for internode (e.g., East-West) communications/traffic. In some examples, the size of the AAs may be between about 1 GB to 10 GB. As can be seen in FIG. **6**A, dividing the storage pod **645** into AAs allows available storage space associated with any given disk or any RAID group to be use across many/all nodes in the cluster without creating silos of space in each node. For example, at the granularity of an individual AA, available storage space within the storage pod **645** may be assigned to any given node in the cluster (e.g., by way of the given node's DEFS(s)). For example, in the context of FIG. **6**A, AA **641***a* and the other AAs shaded in light gray are currently assigned to (or owned by) data DEFS **620***a* (which has a corresponding light gray shading). Similarly, AA **641***b* and the other AAs shaded in dark gray are currently assigned to (or owned by) data DEFS **620***b* (which has a corresponding light gray shading).

[0124] At block **663**, ownership of the AAs is assigned to the DEFSs of the nodes of the cluster. According to one embodiment, an effort may be made to assign group of consecutive AAs to each DEFS. Initially, the distribution of storage space represented by the AAs assigned to each type of DEFS (e.g., data versus log) may be equal or roughly equal. Over time, based on differences in storage consumption by associated workloads, for example, due to differing write patterns, ownership of AAs may be transferred among the DEFSs accordingly.

[0125] As a result, of creating and distributing the disaggregated storage across a cluster in this manner, all disks and all RAID groups can theoretically be accessed concurrently by all nodes and the issue discussed with reference to FIG. **5** in which the entirety of any given disk and the entirety of any given RAID group is owned by a single node is avoided.

Example Context Data Checking Issues Arising in Connection with File Movement

[0126] FIG. **7** is a block diagram conceptually illustrating movement of a volume as well as AA movement and associated metafile movement from a source DEFS **720***a* to a destination DEFS **720***b* in accordance with an embodiment of the present disclosure. Source DEFS **720***a* and destination DEFS **720***b* may be analogous to DEFSs **620***a-b* of FIG. **6**A. Using prior forms of context data, for example, in which certain portions of context data (e.g., a bufftree ID) associated with a file are not unique across the cluster of a storage solution architecture that makes use of distributed storage, for example, similar in nature to that described with reference to FIG. **6**A, may result in false positives indicative of context check mismatches. Consider, for instance, scenarios in which the movement of files (e.g., by way of volume movement, such as performance of a zero-copy volume move or in connection with movement of a metafile or portions thereof associated with an AA being moved as part of a space balancing operation). Subsequent reads from a file that has been moved can result in a false positive indicative of a context mismatch between expected context data associated with a data block read from a file and the actual context data associated with the data block.

[0127] Assuming the use of context data represented by a tuple including a bufftree ID that is only required to be unique within a given DEFS, a data ID (e.g., a file block number (FBN)), and a write time indicator (e.g., a CP count associated with the hosting DEFS file system) that lacks information regarding a cluster-wide timeline, when one or more files (not shown) stored on volume **730***m* are moved (e.g., via a zero-copy volume move operation) from source DEFS **720***a* to

destination DEFS **720***b*, there are scenarios in which false positives and false negatives may arise. First, while the expected context data includes a bufftree ID that is unique within the destination DEFS **720***b*, the source DEFS **720***a* may have assigned a bufftree ID to the moved file that was already in use by the destination DEFS **720***b*, resulting in a bufftree ID collision. Second, due to the independent operation of source DEFS **720***a* and destination DEFS **720***b*, they may perform CPs at different rates. In the context of the present example, source DEFS **720***a* is shown having a CP count of X, whereas destination DEFS **720***b* has a CP count of Y. If the moved file was written during a CP count that is greater than the CP count of the destination DEFS **720***b*, when a read of the moved file is performed, a timeline sanity check (e.g., to confirm the data at issue was not written in the future) based on expected context data utilizing the current CP count of the destination DEFS **720***b* and the context data associated with a block of data read from the moved file will fail.

[0128] As described further below with reference to FIG. **9**, in order to address the issue of bufftree ID collisions when files are moved from one DEFS to another, embodiments described herein ensure when files are created they are assigned respective bufftree IDs that are unique cluster wide.

[0129] According to one embodiment, in order to facilitate proper functioning of cluster-wide timeline checks, a new write time indicator (i.e., an epoch value) is used in place of CP counts. In one embodiment, an epoch value may be maintained for each DEFS in a manner similar to the CP counts (e.g., by incrementing the epoch value each time a CP is performed by the corresponding DEFS). Additionally, when a file movement between a given source DEFS (e.g., source DEFS **720***a*) and a given destination DEFS (e.g., destination DEFS **720***b*) occurs, the epoch value of the destination DEFS is updated to the maximum of the current epoch value of the source DEFS and the current epoch value of the destination DEFS plus one. In this manner, subsequent timeline checks associated with a context check for a read of a moved file will succeed.

Example Data Block

[0130] FIG. **8**A is a block diagram illustrating a data block **800** having context data **820** that may be stored in a physical storage location (e.g., a PVBN) in accordance with an embodiment of the present disclosure. In the present example, the data block **800** is shown including data **805**, a checksum **810**, and the context data **820** of the data block **800**. Data **805** may represent the store data associated with the data block **800**. The checksum **810** may represent a conventional checksum value, for example, in which a sum of all byte values of the data, with any overflow (from carry operations) are discarded.

[0131] According to one embodiment, the context data **820** may include a bufftree ID field **822**, a data ID field **824**, and a write time field **826**. For instance the bufftree ID field **822** may contain a cluster-wide unique bufftreee ID, which may be generated for the file by the particular volume that created/wrote/stored the data block **800** as described with reference to FIG. **9** below. A non-limiting example of a representation of a bufftree ID for a client data file is described further below with reference to FIG. **8**B. A non-limiting example of a representation of a bufftree ID that may be used for metafiles (e.g., an active map, space map, a refecount map, a summary map, etc.) is described further below with reference to FIG. **8**C.

[0132] The data ID field **824** may contain the data ID used by the particular volume to reference (or point to) the data block **800**. For example, the data ID may be a file block number (FBN) used within the file which in turn represents a VVBN within the flexible volume.

[0133] According to one embodiment, the write time field **826** may contain information indicative of a relative time (e.g., an epoch value) at which the data block **800** was written/stored that facilitates performance of cluster-wide timeline checks. In one example, the epoch may be stored as a 32-bit field. Assuming CPs occur at one-second intervals, it would require approximately 126 years to rollover the 32-bit epoch value. Even taking into consideration the epoch jumps, which might occur responsive to file movement from one DEFS to another DEFS, the jumping factor is generally controlled by the fasting moving DEFS in the cluster and is not expected to significantly

decrease the expected rollover time of the epoch value.

Example Bufftree ID for Data Files

[0134] FIG. **8**B is a block diagram illustrating an example of a bufftree ID (e.g., bufftree ID **830**) for a file in accordance with an embodiment of the present disclosure. Bufftree ID **830** may be analogous to bufftree ID **822** of FIG. **8**A. In this example, the bufftree ID is represented by a combination of a DEFS ID **835** and a counter value **837**. The DEFS ID may be a cluster-wide unique value (e.g., of 16 bits) associated with the DEFS hosting the volume within which the associated file is stored. The counter value **837** may be a monotonically increasing volume counter representing the number of volumes created within the particular DEFS. A separate volume counter may be maintained for each DEFS in the cluster. By combining these two values, for example, by prepending or appending the cluster-wide unique DEFS ID **835** to the counter value **837**, the resulting bufftree ID is ensured to also be unique across the cluster. According to one embodiment, the counter value **837** may be represented as a 40-bit value, which is thought to provide a sufficient number of counter values to avoid the need for tracking of freed volume ID values and reuse of same.

[0135] According to one embodiment, the bufftree ID **830** is an extension of a bufftree ID previously used in connection with a storage solution implementing node-level aggerates in which the previously used bufftree IDs are extended by appending the DEFS ID of the DEFS hosting the volume on which a given file is created, in which the DEFS ID is known to be unique across the cluster.

[0136] There are various mechanisms for ensuring cluster-wide uniqueness of IDs assigned to DEFSs within a cluster. According to one embodiment, the DEFS ID for each DEFS in the cluster may be ensured to be unique by performing real-time, cross node interactions at the time DEFSs are created within the cluster. For example, the nodes of the cluster may access a database storing those DEFS ID that have already been used or the nodes may share with each other their respective last used DEFS IDs so as to allow the node to establish a cluster-wide unique DEFS ID for any newly created DEFS. Given DEFS creation is expected to be a relatively rare event, for example, compared to file creation and likely would take place during a storage system initialization process, the cross node communication is not expect to have impact on client interactions with the distributed storage system.

Example Bufftree ID for Metafiles

[0137] FIG. **8**C is a block diagram illustrating an example of a bufftree ID (e.g., bufftree ID **840**) for a metafile in accordance with an embodiment of the present disclosure. Bufftree ID **840** may be analogous to bufftree ID **822** of FIG. **8**A. In this example, the bufftree ID for metafiles (e.g., active map, space map, refcount map, etc.) is represented by a combination of a reserved value **845** and a counter value **847**. In one embodiment, all metafiles will have the same reserved value **845** and this reserved value **845** will not be used as an ID for any DEFSs in the cluster. The reserved value **845** may be the maximum value (e.g., UINT16_MAX) supported by the DEFS ID field **835**. Similarly, the counter value **847** may be a fixed predetermined or configurable counter value for each type of metafile. For instance, in one embodiment, active map metafiles (e.g., active maps **621***a*-*b*) may have a fixed counter value of 0x100 and the space map metafile may have a fixed counter value of 0x200. In this manner, as with client data files, metafiles will be associated with the same bufftree ID as they are moved across the cluster from one DEFS to another.

Example Volume Creation

[0138] One potential mechanism for creating bufftree IDs that are unique across the cluster would be to perform real-time interactions among the nodes (or DEFSs) of the cluster during volume creation. Notably, however, volume creation is a relatively common operation and performing cross-node interactions during volume creation would increase latency experienced by clients of the distributed storage system. As noted above, in one embodiment, generation of cluster-wide unique bufftree IDs relies on at least one portion (e.g., the DEFS ID **835**) that is prepended or

appended to a counter value (e.g., counter value **837**) being cluster-wide unique. In one embodiment, by establishing the cluster-wide unique portion (e.g., the DEFS ID **835**) for a given DEFS at the time the DEFS is created, cross-node interactions associated with bufftree ID generation can be avoided during the volume creation process.

[0139] FIG. **9**A is a flow diagram illustrating operations for performing volume creation in accordance with an embodiment of the present disclosure. The processing described with reference to FIG. **9**A may be performed by a storage system (e.g., node **111***a*, **110***b*, **610***a*, **610***b*, **710***a*, or **710***b*) of a distributed storage system (e.g., cluster **100** or a cluster including nodes **710***a*, **710***b*, and possibly one or more other nodes). In this example, it is assumed a client (e.g., client **180**) of the distributed storage system would like to create a new volume (e.g., one of volumes **730***a-m* and volumes **730***n-x*) within a given DEFS (e.g., DEFS **710***a* or **720***b*) of a node that is serving data access requests on behalf the client.

[0140] At block **910**, a request is received from a client to create a new volume.

[0141] At block **920**, a new volume container is created for the new volume. In one embodiment, volume IDs are inode numbers that correspond to respective inodes of the file system. In such an example, a new inode may be created within the file system for the new volume container and its corresponding inode number may represent the volume ID.

[0142] At block **930**, a cluster-wide, unique bufftree ID is generated for the new volume. According to one embodiment, generation of the bufftree ID includes incrementing a monotonically increasing volume counter maintained for the DEFS that will host the new volume. Then, the new volume counter may be combined with the ID of the DEFS. For example, the DEFS ID may be appended or prepended to the new volume counter value as described above with reference to FIG. **8**B.

[0143] At block **940**, an association between the bufftree ID and the volume is persisted. According to one embodiment, bufftree IDs generated for each new volume are persisted in the disk inode for that volume in the hosting DEFS or otherwise mapped to the disk inode for that volume.

[0144] At block **950**, the volume ID may be returned to the client.

Example File Creation

[0145] FIG. **9**B is a flow diagram illustrating operations for performing file creation in accordance with an embodiment of the present disclosure. As above, the processing described with reference to FIG. **9**B may be performed by a storage system (e.g., node **110***a*, **110***b*, **610***a*, **610***b*, **710***a*, or **710***b*) of a distributed storage system (e.g., cluster **100** or a cluster including nodes **710***a*, **710***b*, and possibly one or more other nodes). In this example, it is assumed the DEFS (e.g., DEFS **710***a* or **720***b*) hosts a volume (e.g., one of volumes **730***a-m* and volumes **730***n-x*) in which a client (e.g., client **180**) of the distributed storage system would like to create a new file.

[0146] At block **960**, a request is received from a client to create a new file within a volume.

[0147] At block **970**, a new file handle is created for the new file. As noted above, in one embodiment, file handles are inode numbers that correspond to respective inodes of the file system. In such an example, a new inode (e.g., inode **435**) may be created within the file system (e.g., within the volume container file of the volume) and its corresponding inode number may represent the file handle.

[0148] At block **980**, an association between a bufftree ID (e.g., in the form of bufftree ID **830**) associated with the volume and the file handle is persisted. As noted above with reference to FIG. **9**A, according to one embodiment, the bufftree ID may be created at the time of volume creation and may be persisted in the disk inode for the volume. In one embodiment, an association may be created between the bufftree ID and the file handle by storing the bufftree ID associated with the volume with which the file is associated within metadata information of the file. For example, the bufftree ID associated with the volume may be copied into a bufftree ID field (e.g., bufftree ID **432**) of the inode (e.g., inode **435**) representing the container for the file along with other context data (e.g., context data **820**) described herein. Similarly, on subsequent writes to the file, the context data may be associated with each block of data of the file.

[0149] At block **990**, the file handle is returned to the client.
[0150] While in the context of FIG. **9**B, the creation of a client data file is described, it is to be appreciated a similar process, albeit, initiated by a workflow or subsystem of the distributed storage system may be used to create metafiles used by the file system; however, the respective bufftree IDs associated with such metafiles may be in the form described with reference to FIG. **8**C.
Example Context Checking
[0151] FIG. **10** is a flow diagram illustrating operations for performing context checking in accordance with an embodiment of the present disclosure. The processing described with reference to FIG. **10** may be performed by a storage system (e.g., node **110***a*, **110***b*, **610***a*, **610***b*, **710***a*, or **710***b*) of a distributed storage system (e.g., cluster **100** or a cluster including nodes **710***a*, **710***b*, and possibly one or more other nodes).
[0152] At block **1010**, a read request is received from a requestor. Depending on whether the file is a client data file or a metafile used by the file system, the requestor may be an external client (e.g., client **180**) of the distributed storage system or an internal workflow or subsystem of the distributed storage system. For example, metafiles may be read by various workflows and/or internal subsystems of the distributed storage system including, but not limited to, write allocation, block free, storage efficiency, and block reallocation.
[0153] At block **1020**, a first set of context data is determined. The first set of context data may represent a set of context data that is expected to be associated with the read request (e.g., based on information specified within the read request, such as the file handle and starting offset of the read and/or based on information associated with the DEFS (e.g., the current epoch value of the DEFS) hosting the volume on which the file is stored. According to one embodiment, the first set of context data may be represented as a tuple (e.g., <bufftree ID, data ID, and epoch>). The expected context may be determined based on the file handle associated with the read request. For example, the hosting volume may be identified based on the file handle, and based on the hosting volume, the bufftree ID and epoch may be identified, Furthermore, from the indirect blocks of the file bufftree the data ID may be determined (e.g., based on VVBN, which represents the FBN).
[0154] At block **1030**, a data block associated with the read request may be read from a cache having one or more levels or from storage as appropriate. For example, the data block may be retrieved from a first-level cache or the buffer cache (if the data block at issue is present in the buffer cache); otherwise, the data block may be read from a second-level cache or the victim cache to which buffer cache entries are evicted (if the data block at issue is present in the victim cache). Finally, if the data block is not present within the cache, the data block may be read from storage.
[0155] At block **1040**, the first set of context data (which may also be referred to herein as the expected context data) is compared with a second set of context data (e.g., context data **820**) associated with (or stored within) the data block at issue. This comparison may be referred to herein as a context check. As noted above, in order to avoid returning stale data to the requestor as a result of a lost write or reallocation of the data block at issue, a context check is performed verify the second set of context data (which may also be referred to as the actual context data as it is the context data actually associated with the data block retrieved) matches or otherwise has an expected relationship with the expected context data. According to one embodiment, the context check involves (i) comparing the bufftree ID of the expected context data with the bufftree ID of the actual context data; (ii) comparing the data ID or FBN with the VVBN of the volume used to reference the data block; and performing a timeline check based on the epoch of the expected set of context data and the epoch contained within the actual context data.
[0156] At decision block **1050**, it is determined whether the context check is confirmed (or satisfied). If not, processing branches to block **1060**. If so, processing continues with block **1090**. In one embodiment, the context check is confirmed (or satisfied) when the bufftree ID comparison indicates a match, the data ID comparison indicates a match, and the timeline check indicates the data was not written after the current epoch of the DEFS (e.g., the epoch value of the actual context

data is less than or equal to the epoch value of the expected context data).
[0157] At block **1060**, RAID recovery may be attempted. For example, an attempt may be made by a RAID subsystem of the distributed storage system to reconstruct the data based on parity information.
[0158] At decision block **1070**, it is determined whether the RAID recovery of the data was successful and the reconstructed data satisfies the context check (described above with reference to blocks **1040** and **1050**). If so, processing continues with block **1090**; otherwise, processing branches to block **1080**.
[0159] At block **1080**, appropriate error processing may be performed, which may include one or more of alerting an administrative user of the distributed storage system regarding the read error and initiating or suggesting performance of a RAID reconstruction process.
[0160] At block **1090**, the read data block or the recovered data block (as the case may be) is returned to the requestor.
[0161] While in the context of FIG. **10**, reading and context checking for a single data block is described, it is to be appreciated blocks **1020** to **1090** may be repeated for each data block associated with the read request.
[0162] While in the context of the flow diagrams of FIGS. **6**B and **9**A-**10** a number of enumerated blocks are included, it is to be understood that examples may include additional blocks before, after, and/or in between the enumerated blocks. Similarly, in some examples, one or more of the enumerated blocks may be omitted and/or performed in a different order.
[0163] Embodiments of the present disclosure include various steps, which have been described above. The steps may be performed by hardware components or may be embodied in machine-executable instructions, which may be used to cause one or more processing resources (e.g., one or more general-purpose or special-purpose processors) programmed with the instructions to perform the steps. Alternatively, depending upon the particular implementation, various steps may be performed by a combination of hardware, software, firmware and/or by human operators.
[0164] Embodiments of the present disclosure may be provided as a computer program product, which may include a non-transitory machine-readable storage medium embodying thereon instructions, which may be used to program a computer (or other electronic devices) to perform a process. The machine-readable medium may include, but is not limited to, fixed (hard) drives, magnetic tape, floppy diskettes, optical disks, compact disc read-only memories (CD-ROMs), and magneto-optical disks, semiconductor memories, such as ROMs, PROMs, random access memories (RAMs), programmable read-only memories (PROMs), erasable PROMs (EPROMs), electrically erasable PROMs (EEPROMs), flash memory, magnetic or optical cards, or other type of media/machine-readable medium suitable for storing electronic instructions (e.g., computer programming code, such as software or firmware).
[0165] Various methods described herein may be practiced by combining one or more non-transitory machine-readable storage media containing the code according to embodiments of the present disclosure with appropriate special purpose or standard computer hardware to execute the code contained therein. An apparatus for practicing various embodiments of the present disclosure may involve one or more computers (e.g., physical and/or virtual servers) (or one or more processors (e.g., processors **222***a*-*b*) within a single computer) and storage systems containing or having network access to computer program(s) coded in accordance with various methods described herein, and the method steps associated with embodiments of the present disclosure may be accomplished by modules, routines, subroutines, or subparts of a computer program product.
[0166] The term “storage media” as used herein refers to any non-transitory media that store data or instructions that cause a machine to operation in a specific fashion. Such storage media may comprise non-volatile media or volatile media. Non-volatile media includes, for example, optical, magnetic or flash disks, such as storage device (e.g., local storage **230**). Volatile media includes dynamic memory, such as main memory (e.g., memory **224**). Common forms of storage media

include, for example, a flexible disk, a hard disk, a solid state drive, a magnetic tape, or any other magnetic data storage medium, a CD-ROM, any other optical data storage medium, any physical medium with patterns of holes, a RAM, a PROM, and EPROM, a FLASH-EPROM, NVRAM, any other memory chip or cartridge.

[0167] Storage media is distinct from but may be used in conjunction with transmission media. Transmission media participates in transferring information between storage media. For example, transmission media includes coaxial cables, copper wire and fiber optics, including the wires that comprise bus (e.g., system bus **223**). Transmission media can also take the form of acoustic or light waves, such as those generated during radio-wave and infra-red data communications.

[0168] Various forms of media may be involved in carrying one or more sequences of one or more instructions to the one or more processors for execution. For example, the instructions may initially be carried on a magnetic disk or solid state drive of a remote computer. The remote computer can load the instructions into its dynamic memory and send the instructions over a telephone line using a modem. A modem local to the computer system can receive the data on the telephone line and use an infra-red transmitter to convert the data to an infra-red signal. An infra-red detector can receive the data carried in the infra-red signal and appropriate circuitry can place the data on bus. Bus carries the data to main memory (e.g., memory **224**), from which the one or more processors retrieve and execute the instructions. The instructions received by main memory may optionally be stored on storage device either before or after execution by the one or more processors.

[0169] All examples and illustrative references are non-limiting and should not be used to limit the applicability of the proposed approach to specific implementations and examples described herein and their equivalents. For simplicity, reference numbers may be repeated between various examples. This repetition is for clarity only and does not dictate a relationship between the respective examples. Finally, in view of this disclosure, particular features described in relation to one aspect or example may be applied to other disclosed aspects or examples of the disclosure, even though not specifically shown in the drawings or described in the text.

[0170] The foregoing outlines features of several examples so that those skilled in the art may better understand the aspects of the present disclosure. Those skilled in the art should appreciate that they may readily use the present disclosure as a basis for designing or modifying other processes and structures for carrying out the same purposes and/or achieving the same advantages of the examples introduced herein. Those skilled in the art should also realize that such equivalent constructions do not depart from the spirit and scope of the present disclosure, and that they may make various changes, substitutions, and alterations herein without departing from the spirit and scope of the present disclosure.

Claims

1. A non-transitory machine readable medium storing instructions, which when executed by one or more processing resources of a distributed storage system, cause the distributed storage system to: receive, within a node of a plurality of nodes of a cluster representing the distributed storage system, a read request from a requestor to read data from a file; determine an expected set of context data associated with the read request including at least a first volume identifier (ID) associated with the file that is unique across the cluster and a current epoch value of a dynamically extensible file system (DEFS) in which the file is stored; and prior to returning a block of data of the requested data to the requestor, verify: a second volume ID contained within a second set of context data associated with the block of data matches the first volume ID in the expected set of context data; and an epoch value contained within the second set of context data and the current epoch value of the expected set of context data satisfy a cluster-wide timeline check.

2. The non-transitory machine readable medium of claim 1, wherein the cluster-wide timeline check is satisfied when the epoch value is less than or equal to the current epoch value.

3. The non-transitory machine readable medium of claim 1, wherein the first set of context data further includes a file block number (FBN) based on a starting offset specified by the read request, and wherein the instructions further cause the distributed storage system to verify the FBN matches a virtual volume block number (VVBN) of a volume used to reference the data block or an FBN contained within the second set of context data.
4. The non-transitory machine readable medium of claim 1, wherein the requestor is a client of the distributed storage system.
5. The non-transitory machine readable medium of claim 4, wherein a volume containing the file was hosted by a different DEFS of the node or another node of the plurality of nodes a time at which the file was created and a volume ID associated with the file was generated based on a combination of a cluster-wide, unique ID of the different DEFS and a volume counter value associated with the different DEFS.
6. The non-transitory machine readable medium of claim 4, wherein a volume containing the file was hosted by the DEFS at a time at which the file was created and a volume ID associated with the file was generated based on a combination of a cluster-wide, unique ID of the DEFS and a volume counter value associated with the DEFS.
7. The non-transitory machine readable medium of claim 1, wherein the requestor is a subsystem or workflow of the distributed storage system and wherein the file comprises a metafile containing metadata used by the DEFS.
8. The non-transitory machine readable medium of claim 7, wherein during creation of the metafile, a volume ID was associated with metafile that was previously generated based on a combination of a reserved value and a fixed counter value selected based on a type of the metafile.
9. A method comprising: receiving, by a dynamically extensible file system (DEFS) of a node of a plurality of nodes of a cluster representing a distributed storage system, a read request from a client to read data from a file contained within a volume hosted by the DEFS; determining a first set of context data associated with the read request including at least a first buffer tree identifier (bufftree ID) associated with the file that is unique across the cluster and a current epoch value of the DEFS; and prior to returning a block of data of the requested data to the client, verifying: a second bufftree ID contained within a second set of context data associated with the block of data matches the first bufftree ID; and an epoch value contained within the second set of context data and the current epoch value satisfy a cluster-wide timeline check.
10. The method of claim 9, wherein the cluster-wide timeline check is satisfied when the epoch value is less than the current epoch value.
11. The method of claim 9, wherein the first set of context data further includes a file block number (FBN) based on a starting offset specified by the read request, and wherein the method further comprises verifying the FBN matches an FBN contained within the second set of context data.
12. The method of claim 1, wherein a volume containing the file was hosted by a different DEFS of the node or another node of the plurality of nodes a time at which the file was created and a bufftree ID associated with the file was generated based on a combination of an ID of the different DEFS and a volume counter value associated with the different DEFS.
13. The method of claim 1, wherein a volume containing the file was hosted by the DEFS at a time at which the file was created and a bufftree ID associated with the file was generated based on a combination of an ID of the DEFS and a volume counter value associated with the DEFS.
14. A distributed storage system comprising: one or more processing resources; and instructions that when executed by the one or more processing resources cause the distributed storage system to: receive, within a node of a plurality of nodes of a cluster representing the distributed storage system, a read request from a requestor to read data from a file; determine an expected set of context data associated with the read request including at least a first buffer tree identifier (bufftree ID) associated with the file that is unique across the cluster and a current epoch value of a dynamically extensible file system (DEFS) in which the file is stored; and prior to returning a block

of data of the requested data to the requestor, verify: a second bufftree ID contained within a second set of context data associated with the block of data matches the first bufftree ID in the expected set of context data; and an epoch value contained within the second set of context data and the current epoch value of the expected set of context data satisfy a cluster-wide timeline check.

15. The distributed storage system of claim 14, wherein the cluster-wide timeline check is satisfied when the epoch value is less than or equal to the current epoch value.

16. The distributed storage system of claim 14, wherein the first set of context data further includes a file block number (FBN) based on a starting offset specified by the read request, and wherein the instructions further cause the distributed storage system to verify the FBN matches a virtual volume block number (VVBN) of a volume used to reference the data block or an FBN contained within the second set of context data.

17. The distributed storage system of claim 14, wherein the requestor is a client of the distributed storage system.

18. The distributed storage system of claim 17, wherein a volume containing the file was hosted by a different DEFS of the node or another node of the plurality of nodes a time at which the file was created and a bufftree ID associated with the file was generated based on a combination of a cluster-wide, unique ID of the different DEFS and a volume counter value associated with the different DEFS.

19. The distributed storage system of claim 17, wherein a volume containing the file was hosted by the DEFS at a time at which the file was created and a bufftree ID associated with the file was generated based on a combination of a cluster-wide, unique ID of the DEFS and a volume counter value associated with the DEFS.

20. The distributed storage system of claim 14, wherein the requestor is a subsystem or workflow of the distributed storage system, wherein the file comprises a metafile containing metadata used by the DEFS, and wherein during creation of the metafile, a bufftree ID was associated with metafile that was previously generated based on a combination of a reserved value and a fixed counter value selected based on a type of the metafile.